

**CARRT Assistive Robotic Arm
Final Project Deliverables**

CIS 4910 Computer Science / Engineering Project
Spring 2026

Company / Customer:
CARRT Lab, University of South Florida

Students:
Oliver Gaston
Brielle Ashmeade
Allison Moline
Jadon Orr

Department of Computer Science and Engineering
University of South Florida

Requirements for an ADL Assistive Robotic Arm

**Oliver Gaston
Brielle Ashmeade
Allison Moline
Jadon Orr**

Department of Computer Science and Engineering
University of South Florida
Tampa, FL 33620

Email: gaston13@usf.edu
ashmeadeb@usf.edu
allison439@usf.edu
jjorr@usf.edu

Version 2.0
Date: February 19, 2026

History of document

- **Version 1.00** (February 6, 2026) – File Created
- **Version 2.00** (February 19, 2026) – Refined with Professor's Instruction
 - Updated sections 1-4 to improve specificity and measurability, using Dr. Christensen's feedback.

Table of Contents

1. Introduction	1
2. Glossary	1
3. Assumptions	1
4. Requirements	1
5. Needs and factors	2
5.1 Public health needs	2
5.2 Public safety needs	2
5.3 Public welfare needs	2
5.4 Global factors	3
5.5 Cultural factors	3
5.6 Social factors	3
5.7 Environmental factors	3
5.8 Economic factors	3
References	3

1. Introduction

The problem addressed by these requirements is:

“Identify and implement three unique Activities of Daily Living (ADLs) that can be automated by a Kinova Gen3 robotic arm (in real-time) mounted on their wheelchair, specifically for a user with a physical disability affecting upper-limb mobility.

Specifically, create a system that can:

- clear household objects from a table surface,
- retrieve a fallen water bottle within the arm’s range, and
- support medication administration through identity confirmation and retrieval of a medicine bottle.”

2. Glossary

Activity of Daily Living: ADL; activities essential to everyday life that are often made more difficult when living with a disability.

Household Object: a simple rigid object common in daily human life, grabbable by the Kinova Gen3 arm, and with an attached QR code with a unique identification code for each object type. (eg. small box, plastic bottle, cup, medicine bottle). This object should have a bounding box size between about 5 and 15cm, and be lighter than 1kg.

Kinova Gen3 Robotic Arm: A lightweight robotic arm designed to safely and precisely pick up and move objects. For the project, the arm is assumed to be attached to one side of a user’s wheelchair, for regular use.

Medicine Bottle: a rigid cylindrical pill bottle with a printed label listing a patient’s name clearly, and a unique QR code for robot identification.

Physical Disability: An impairment that majorly limits mobility, physical functioning, or dexterity in an individual’s daily life.

Real-Time: updates to object poses should occur at least 5 times per second, less than 100-200 ms.

Table: a flat, horizontal surface, with a height of approximately 45cm, and dimensions of 55cm × 55 cm, existing within the Kinova arm’s workspace.

Upper-Limb Mobility: Ease of movement in arms, forearms, wrists and hands.\

Water Bottle: a standard waterbottle (eg. 500-750 mL), with a mass less than 1kg, lying on its side on the floor.

Wheelchair: a medical device used by a person with a physical disability that affects their mobility, for regular, daily locomotion.

3. Assumptions

The assumptions for this project are:

- 1) The Kinova Gen3 robotic arm (and attached camera) is attached to a wheelchair and operates within a limited, predefined workspace reachable by the user, including a table, a region of floor, and (possibly) a medication shelf.
- 2) The user is a wheel-chair seated individual with significantly reduced upper-limb mobility but can operate and interact with a simple UI.
- 3) The user has the ability to give voice commands to the robotic arm through a simple user interface, or by pressing large buttons on a tablet interface.
- 4) The ADLs that the robotic arm assists with are object-manipulation tasks and not full-body mobility tasks.
- 5) Interactable objects have QR markers visible on them, in order to assist with robust perception by the robotic arm.

4. Requirements

- 1) As a daily user, I want to select ADL tasks with an accessible interface, so that I can control the arm independently.
 - a) The system should provide a tablet UI that allows a user to use either voice or buttons to select three ADLs: Clear Table, Pick Up Bottle, Medication Hand-Off
 - b) The UI should not require more than two user actions to start a task (activate listening + one voice command, or one button press).
 - c) The UI should display the current task status with clear text or icons (eg. Idle, Executing, Done, Error, Cancel) and update within 2 seconds of a state change.
- 2) As a daily user, I want the robotic arm to clear light household objects from a table, so that the surface can be used for another activity.
 - a) The system should identify at least 3 items located on top of a table's surface, within the Kinova Gen3 arm's workspace.
 - b) For each identified object, the system should plan and execute a motion to:
 - i) Grasp the item, and
 - ii) Move the item to its predefined drop location (eg. bin / shelf) within the workspace.
 - c) In at least 80% of simulation runs, the system should clear at least 3 items in one execution of the ADL without collisions with the table, robot arm, or wheelchair.
 - d) During real trials, the system should complete the ADL with at least 70% accuracy in moving all selected items, without collisions.
- 3) As a daily user, I want the robotic arm to pick up a fallen water bottle from the floor, so that I can easily retrieve it if dropped.
 - a) The system should detect the pose of the water bottle on the floor, as defined in the Glossary, lying within the workspace of the arm.
 - b) The system should grasp the water bottle and lift it to tabletop height about the floor.
 - c) The system should then place the bottle at a specified location on the table, such as near the edge.

- d) In simulation, the ADL should succeed in detecting, picking up, and placing the bottle in at least 80% of trials.
 - e) In real tests, the ADL should succeed in at least 70% of runs without collisions with the floor, table, wheelchair, or robot self-collisions.
- 4) As a daily user, I want the robotic arm to help with medication administration by handing me the correct bottle, so that I don't need outside assistance to do so.
 - a) The system should detect and identify a medication bottle on a shelf or table, using its QR code, and move to read the label.
 - b) The UI should display or announce the detected medication and ask the user to confirm their name.
 - c) If the user's name matches the label on the bottle, the system will execute to:
 - i) Grasp the medication bottle, and
 - ii) Move it to a handover location, within the user's reachable area.
 - d) If the medication does not have the user's name, the system should not move the arm, display or announce the error, and end the action.
 - e) In simulation, the system should correctly match and hand off medication in 80% of the runs.
- 5) As a developer, I want the robotic arm to use QR-based perception, so that object and interactable locations are well recognizable.
 - a) QR markers should be clear and visible at short-range distances.
 - b) The correct target should be identified in at least 80% of perception attempts.
 - c) The perception system should operate in real time.
- 6) As a developer, I want to run the operations in simulation first, so that the system behavior is tested safely and repeatedly and real-world collisions are avoided.
 - a) The system should run in ROS2 and Gazebo for simulation, including environment objects (table, floor, medication bottle, shelf).
 - b) All three ADL tasks should be demonstrable in simulation before real testing is implemented.
 - c) The simulation should include any necessary environment objects (table, switch, objects)

5. Needs and factors

For each need and factor describe considerations for this project. Some needs and/or factors may not apply to the project, if so then note as such (and briefly describe why it does not apply).

5.1 Public health needs

While not directly stated in the requirements, public health needs are indirectly addressed by the project, where we are hoping to improve disabled user's daily life through supporting them with ADLs. By reducing the need for disabled users to over-exert themselves, or ask for outside assistance, the system could lower risk of strain or injury. The ADL tasks relevant are in requirements (2), (3), and (4).

5.2 Public safety needs

Public safety is driven by requirement (6). This requirement describes the need for simulation and not simply running tests and otherwise on the real equipment. This ensures the safety of developers and future users.

5.3 Public welfare needs

Public welfare, similarly to public health, is indirectly addressed, through project goals of increasing user independence. Through assistance with ADLs, (in requirements (2), (3), and (4)) the robotic arm works to improve the quality of life for disabled individuals.

5.4 Global factors

Global factors are not addressed by the project, as the goal is more localized. The system doesn't involve global effects, since it is only applicable as an assistive robot for those with disabilities.

5.5 Cultural factors

Cultural factors are also not addressed by this project. The robotic arm works to provide basic assistance in daily tasks present over the world, and typically unaffected by a user's culture.

5.6 Social factors

Social factors are addressed through the focus on the goal of independence and improved accessibility. The arm supports users in everyday tasks that they'd often otherwise need assistance for, reducing social dependency and improving user confidence and regular relations in shared spaces. This is directly mentioned in requirement (4), where the goal is a lack of outside assistance, though this can be applied to the other two ADL tasks as well.

5.7 Environmental factors

The robotic arm does not generate significant waste, so the project is only minimally related to environmental factors. It remains limited to local electricity/hardware usage, which are not within the scope of the project.

5.8 Economic factors

Economic factors are indirectly mentioned in Requirement (6). Through the use of simulation software, and through that software being open-source, the cost of development dramatically is reduced. Otherwise, since we were provided the hardware, cost was decently low already. Also, while cost analysis lies outside the project scope, there is potential in assistive robotic technology to improve long-term caregiving costs.

References

- [1] "Assistive robotics for activities of daily living - do-it," DO,
<https://doit.uw.edu/programs/accessengineering/accessible-design-of-engineered-products-and->

technology-adept/adept-accessibility-briefs/assistive-robotics-for-activities-of-daily-living/
(accessed Feb. 6, 2026).

- [2] T. L. Mitzner, T. L. Chen, C. C. Kemp, and W. A. Rogers, “Identifying the potential for robotics to assist older adults in different living environments,” *International journal of social robotics*, <https://pmc.ncbi.nlm.nih.gov/articles/PMC3979567/> (accessed Feb. 6, 2026).

Specification: CARRT Robotic Assistive Arm Project

Oliver Gaston
Brielle Ashmeade
Allison Moline
Jadon Orr

Bellini College of AI, Cybersecurity and Computing
University of South Florida
Tampa, FL 33620

Email:
gaston13@usf.edu
ashmeadeb@usf.edu
allison439@usf.edu
jjorr@usf.edu

Version 4.0
April 27, 2026

History of Specification Document

- **Version 1.00** (March 3, 2026) – Initial document created
- **Version 2.00** (March 5, 2026) – Added to document sections: Design, Constraints, and Tradeoffs.
- **Version 3.00** (March 6, 2026) – Completed and refined document for submission.
- **Version 4.00** (April 27, 2026) – Edited trade-offs to more clearly show decision-making and option weightings.

Table of Contents

1. Introduction	1
2. Glossary	1
3. Constraints	2
4. Applicable standards	3
5. Design	4
6. Risk analysis and mitigation	9
7. Design trade-offs.....	10
8. Project plan	12
9. Traceability to requirements	13
10. Traceability to needs and factors	15
References	17

1. Introduction

The purpose of this document is to describe the design specifications for the CARRT Assistive Arm Project. The main goal of this project is to develop an assistive robotic system capable of helping individuals with limited upper-limb mobility perform Activities of Daily Living (ADLs).

The system uses a Kinova Gen3 robotic arm mounted on a wheelchair and controlled through a ROS 2-based software architecture, with an integrated camera that allows the arm to see and interact with its environment. A tablet-based user interface allows the user to issue commands either through voice input or large touch buttons designed for accessibility. It addresses the following problem statement, from our previous Requirements:

Identify and implement three unique Activities of Daily Living (ADLs) that can be automated by a Kinova Gen3 robotic arm mounted on a wheelchair, specifically for a user with a physical disability affecting upper-limb mobility. The three tasks are: (1) clear household objects from a table surface, (2) retrieve a fallen water bottle within the arm's range, and (3) support medication administration through identity confirmation and retrieval.

Object detection is performed using AprilTag markers attached to objects, allowing the system to identify and locate objects within the robot's workspace. The system is first implemented and tested in simulation using ROS2 and Gazebo before being deployed on the physical hardware of the robotic arm. We work first to achieve an 80% success rate in simulation before we move to real-world testing.

This specification document describes the modular system architecture, user interface, perception system, motion planning approach, and development plan used to implement the assistive robotic system.

2. Glossary

Action Client: a ROS2 action instance, to send a typed goal message to a server, receive feedback during execution, and retrieve a result after execution. Used in `helper_moves` to interface with the MoveGroup action server at `/move_action`.

Activity of Daily Living: ADL; an activity essential to everyday life that are often made more difficult when living with a disability.

AprilTag: a QR code tag that connects to vision package `tag36h11`, allows for the connection to ROS2 to connect and "see" visible interactable objects. [3]

Base_link: the root coordinate frame of the Kinova Gen3 arm, fixed at the center of the arm's mounting position. All object poses, collision objects, and motion goals in the system are expressed in this frame.

Collision Object: A ROS2 message defining a geometric primitive (box, cylinder, sphere) with a pose, in a given coordinate frame. Used to plan obstacles for the arm to avoid during planning.

GetTagPose.srv: A ROS2 service definition in `adl_interfaces/srv/`. Used to implement a server on a topic name, to track tag poses.

Home Position: a robot configuration in the Kinova Gen3 SRDF file, under the group name “manipulator”. Used as a safe resting position before and after tasks, and as recovery in motion failure.

Household Object: a simple rigid object common in daily human life, grabbable by the Kinova Gen3 arm, and with an attached QR code with a unique identification code for each object type. (eg. small box, plastic bottle, cup, medicine bottle). This object should have a bounding box size between about 5 and 15cm and be lighter than 1kg.

Kinova Gen3 Robotic Arm: A lightweight robotic arm designed to safely and precisely pick up and move objects. For the project, the arm is assumed to be attached to one side of a user’s wheelchair, for regular use.

Medicine Bottle: a rigid cylindrical pill bottle with a printed label listing a patient’s name clearly, and a unique QR code for robot identification.

MoveGroup: the central planning and execution server. Accepts MotionPlanRequest goals, performs collision-aware path planning, and executes approved trajectories on robot controllers.

MotionPlanRequest: A message used to define a specific motion planning problem, when considering known obstacles and constraints.

Physical Disability: An impairment that majorly limits mobility, physical functioning, or dexterity in an individual’s daily life.

Planning Scene: the internal MoveIt 2 data structure for all known geometry in the robot’s environment. Published at startup by scene_static and the scene vision node, and regularly updated during completion.

Real-Time: updates to object poses should occur at least 5 times per second, less than 100-200 ms.

Table: a flat, horizontal surface, with a height of approximately 45cm, and dimensions of 55cm × 55 cm, existing within the Kinova arm’s workspace.

Tool Frame: the end-effector’s reference frame, located at the center of the gripper mounting interface.

TF2: the ROS2 transformation library. It maintains a tree of coordinate frame transformations to transform a pose from one frame to another.

Upper-Limb Mobility: Ease of movement in arms, forearms, wrists and hands.

Water Bottle: standard waterbottle (eg. 500-750 mL), with a mass of less than 1kg, lying on its side on the floor.

Wheelchair: a medical device used by a person with a physical disability that affects their mobility, for regular, daily locomotion.

3. Constraints

There are several constraints that influence the design and implementation of the assistive robotic arm system.

1. Schedule

- All development and testing must be completed within the course timeline.

- Simulation milestones must precede real hardware testing.
- No ADL task can be tested on the arm before achieving an 80% success rate in simulation.

2. Hardware

Parameter	Constraint
Robot Arm	Kinova Gen3, 7 DoF, manipulator
Gripper	Robotiq 2F-85, max opening 85 mm (0.085 m)
Gripper Force	5N-220N, from Robotiq 2F-85 datasheet [7]
Gripper Speed	13–101 mm/s, from Robotiq 2F-85 datasheet [7]
Arm Payload	4 kg
Camera	Intel RealSense D415
Camera Resolution	640x480 pixels
Tag Size	5cm x 5cm April Tag
Object Size Range	5-15cm bounding box, <= 1kg

3. Software

- We are working with ROS2 Jazzy on Ubuntu 24.04 LTS.
- MoveIt2 is necessary for motion planning.
- The system must be fully operable in simulation before real hardware testing.
- Simulation uses Gazebo physics engine.

4. Workspace

All interactable objects exist within the arm’s reachable workspace. The lab space is as follows:

Element	Dimensions	Position
Table Surface	$0.55 \times 0.55 \times 0.45$	(0.60, 0.00, 0.225)
Wheelchair Block	$0.05 \times 0.80 \times 0.80$	(-0.30, 0.00, 0.40)
Back Wall	$0.05 \times 1.00 \times 1.20$	(0.95, 0.00, 0.60)
Floor	$2.00 \times 2.00 \times 0.01$	(0.00, 0.00, -0.005)

5. Perception

- AprilTag markers must be on all manipulable objects and destinations.
- Tags must be visible to the camera from the planned approach direction.
- Object bounding boxes must be between 5 and 15cm in any dimension.
- Pose updates occur at 5Hz, with a latency of less than 200 ms (real time, Req 5c)

6. Safety

- All motion plans are checked by MoveIt for collisions against the scene before execution.
- On task failure, the arm must go to home position, for recovery.
- The medication hand-off cannot move unless the user’s name is confirmed.
- Real hardware testing can’t begin until 80% of simulation is successful.

7. Accessibility

- The UI must require no more than two user actions to initiate any task (Req 1b).
- The UI must support both voice commands and button presses (Req 1a).
- Task status must be displayed with text or icons and updated within 2 seconds of state changes (Req 1c).

4. Applicable standards

Standard	Importance to Project
----------	-----------------------

ROS2 REP-103 [6]	Defines standard units of measurement. All numerical values must conform to these units.
ROS2 REP-105 [5]	Defines standard coordinate frames (base_link, map, camera_link, etc.). The base_link is used as the reference for object poses, collision objects, and motion goals.
ISO 10218-1:2011	Defines safety requirements for industrial robots. Informs design requirements for collision avoidance, simulation-first testing strategies, as well as 30% velocity/acceleration limits during ADL tasks.
IEC 62061:2021	Functional safety standard for machinery control systems. Requires collision detection and safe-stops.
Robotiq 2F-85 Manual [7]	Defines the Robotiq gripper's position, force, and speed ranges.

5. Design

This section describes the system's complete design. It is organized from high-level to low-level.

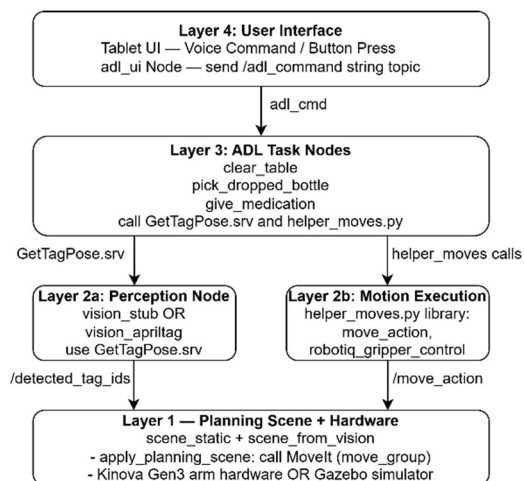


Figure 1: High-Level Project Architecture

fixed and known in advance. All collision object dimensions and positions in section 3.3 were measured from the physical lab setup and are hardcoded into scene_static to be avoided. The arm is mounted to the left side of the wheelchair, and the wrist camera is mounted at the end of link 6, with its optical axis aligned with the tool_frame z axis. This workspace is pictured in Figure 2.

5.1. System Overview

The system is structured into four functional layers, where control flows downward from the user to the actual hardware implementation. Information moves upwards from sensors to the task logic. These layers help make sure that each layer can be individually tested and developed. Pictured left, in Figure 1, is a diagram showcasing the project's functional structure in four layers.

5.2. Hardware Configuration

The physical workspace is

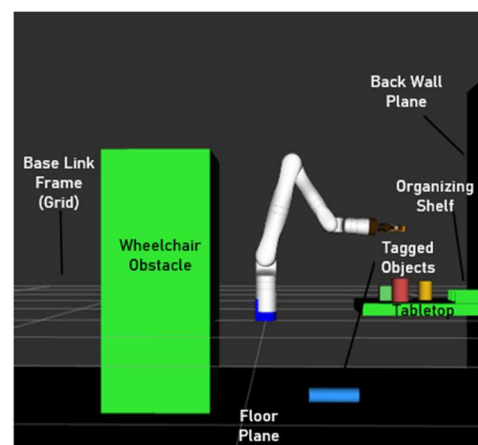


Figure 2: Labeled Robot Workspace

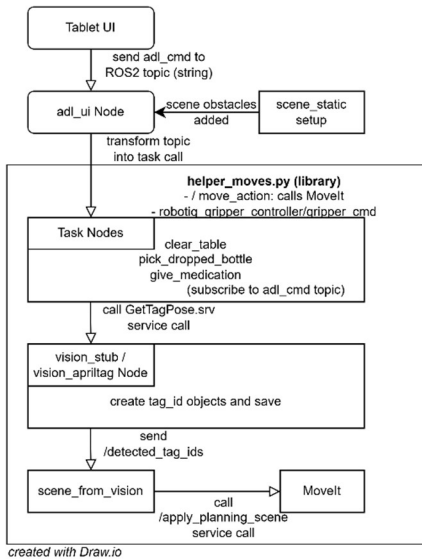


Figure 3: ROS2 Node Flowchart

5.3. Software Architecture

All system components are ROS2 nodes within the `adl_tasks` and `adl_interfaces` packages. `Helper_moves.py` acts as a Python library, rather than a node, to be imported by task nodes when started. Shown in Figure 3.

The two ROS2 packages are structured as follows:

`adl_interfaces` – custom service definition (`GetTagPose.srv`). Built as a cmake package, required for service type generation.

`adl_tasks` – all used Python nodes and libraries

- `vision_stub.py`, `vision_apriltag.py`,
- `scene_static.py`, `scene_from_vision.py`, `helper_moves.py`
- `clear_table.py`, `pick_dropped_bottle.py`, `give_medication.py`
- `adl_ui.py`, `apriltag_key.py`.

Built as Python packages.

Build and Launch Commands:

```
colcon build --symlink-install --packages-select adl_interfaces adl_tasks && source
install/setup.bash
```

```
ros2 launch kinova_gen3_7dof_robotiq_2f_85_moveit_config robot.launch.py
robot_ip:=192.168.0.1 use_fake_hardware:=true
```

Then, each in its own terminal, run:

```
ros2 run adl_tasks scene_static, ros2 run adl_tasks vision_stub, ros2 run adl_tasks clear_table
```

5.4 Interface and Message Specifications

This section specifies the communication channels within the system as defined in our system: topics, services and actions.

5.4.1 Topics

Topic Name	Message Type	Publisher Subscriber	Description
/adl_command	std_msgs/String	adl_ui task nodes	One string defining the specific task to be called. Only triggers execution. Only one task should trigger at once.
/detected_tag_ids	std_msgs/Int32Multi Array	Vision node task nodes, scene_from_vision	Published at 10Hz. Hata field contains a list of currently visible AprilTag IDs. Stale IDs are to be removed after 5s of no detection.
/wrist_mounted_camera/image	sensor_msgs/Image	Camera driver vision_apriltag	Raw color image from RealSense D415. Convert for tag scanning with CV bridge. Published at 30Hz.
/joint_states	sensor_msgs/JointState	Ros2_control all nodes	Current joint positions, velocities, and efforts. Used for MoveIt planning and to verify Home.

/robotiq_gripper_controller/joint_trajectory	trajectory_msgs/JointTrajectory	Helper_moves gripper control	Used in grab_object to control grasp force and speed.
/picked_ids	std_msgs/Int	Task nodes scene_from_vision	Published by task nodes after a successful pick. Removes the object as a collision object to prevent phantom collisions.

5.4.2 Services

Action Name	Type	Request / Response
get_tag_pose	Adl_interfaces/GetTagPose	Request: int32 tag_id (currently 0-8) Response: geometry_msgs/Pose pose, bool success, string message. Server blocks until tag is found or 5 second timeout completes.
/apply_planning_scene	Moveit_msgs/ApplyPlanningScene	Request: moveit_msgs/PlanningScene scene Response: bool success. Called by scene_static and scene from vision.

5.4.3 Actions

Action Name	Type	Goal / Result Used
/move_action	Moveit_msgs/MoveGroup	Goal: MoveGroup.Goal with request and planning options Result: int error code. Value 1 indicates success.
/robotiq_gripper_controller/gripper_cmd	Control_msgs/GripperCommand	Goal: position, max_effort. Result: position, effort, stalled, reached_goal. Used by close_gripper and open_gripper in simulation.

5.4.4 Custom Service Definition

The GetTagPose service is defined in adl_interfaces/srv/GetTagPose.srv as:

```
# GetTagPose.srv
# Request
int32 tag_id          # AprilTag ID to look up (0-8)
---
# Response
geometry_msgs/Pose pose # pose in base_link frame
bool success           # false if tag not found within timeout
string message         # human-readable status or error message
```

5.5 Perception System

The system uses AprilTag (tag36h11) detection to convert raw camera images into 6-DOF object poses in the base_link coordinate frame. The perception system was designed to allow either the stub or the real vision nodes to be tested and to create an identical interface. Task nodes should remain unaware of which is running.

- **INPUT:** /wrist_mounted_camera/image
- **PROCESSING PIPELINE (vision_apriltag):**
 - o 1. Convert ROS image message to OpenCV grayscale numpy array.
 - o 2. Return list of detection objects, each with a tag_id, pose, center, and corners.
 - o 3. For each detection where tag_id is in dictionaries OBJECTS or LOCATIONS:
 - Build the pose, transform to the base_link pose, and cache.
 - o 4. Use a timer at 10Hz to purge old tags and publish remaining.

- 5. GetTagPose service handler: validate tag is in the dictionaries and fail else.
Return pose if found within 5s, or return failure.
- **OUTPUT:** /detected_tag_ids
- **STUB MODE:** returns hardcoded stub poses as always visible, without camera interface.

Tag Placement Specifications

Object	ID	Tag Placement	Approach Vector	Grasp Offset (m)	Gripper Width (m)	Force (N) Speed (m/s)
Water Bottle	0	Side (face-up, on floor)	[0, 0, -1] above	[0.05, 0, 0]	0.067 + tol	15 0.03
Medication	1	Side (upright, facing robot)	[1, 0, 0] horizontal	[0, 0, 0.04]	0.060 + tol	10 0.03
Cup	2	Side (upright, facing robot)	[1, 0, 0] horizontal	[0, 0, 0.04]	0.075 + tol	10 0.05
TV Remote	3	Top (face up)	[0, 0, -1] above	[0, 0, 0.02]	0.043 + tol	10 0.03
Cube	4	Top (face up)	[0, 0, -1] above	[0, 0, 0.05]	0.065 + tol	10 0.05

5.6 Planning Scene Management

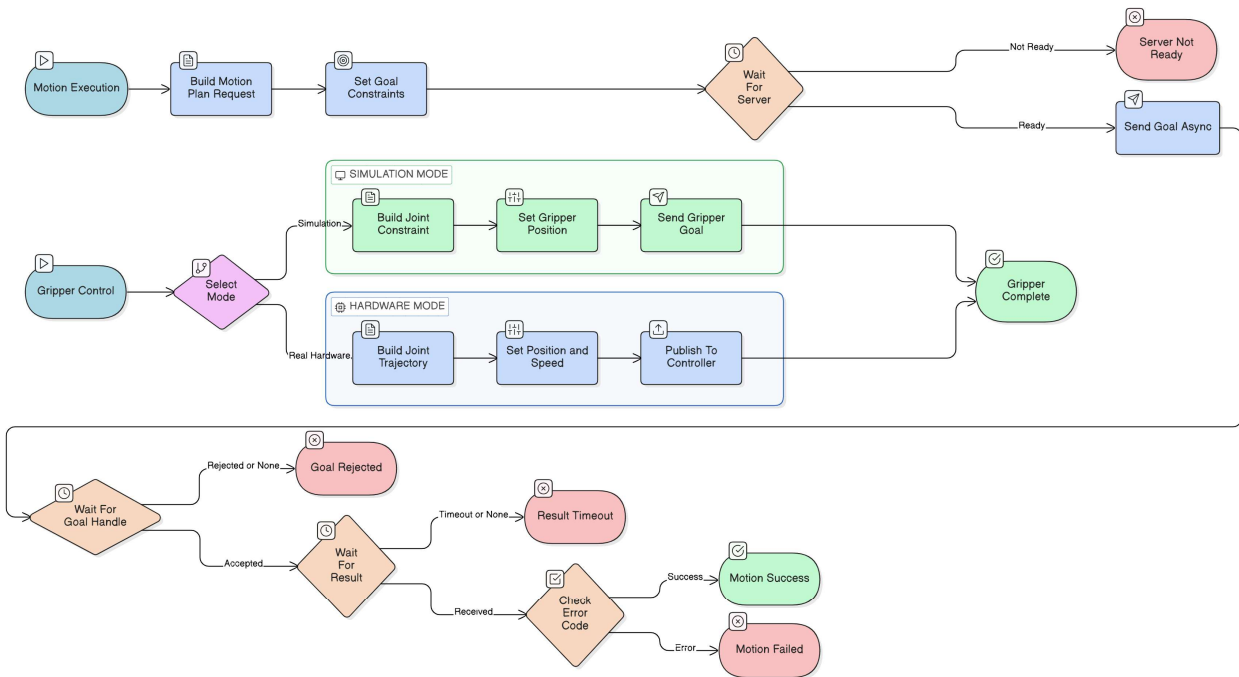
The MoveIt planning scene is the main representation of all obstacles, and is initialized with permanent geometry on startup, with dynamically updated objects as detected and picked up. All motion plans are checked against the planning scene before execution, which would reject any plan that would cause a collision.

5.7 Motion Planning and Execution

Any arm motion is planned by MoveIt 2, through the OMPL planner suite and the MoveGroup action server. Task nodes use helper_moves.py to call MotionPlanRequests and submit them as MoveGroup action goals. Gripper motion then either uses MoveIt states or real hardware through JointTrajectory publication.

The MoveItHelper class is instantiated once per task node and holds an ActionClient, to apply planning parameters to an arm motion goal:

- Group_name: Planning group for the robotic arm, defined in SRDF
- Num_planning_attempts: number of allowed planning attempts
- Allowed_planning_time: time for planning before move
- Max_velocity_scaling_factor: max speed allowed near a user
- Max_acceleration_scaling_factor: max acceleration allowed. Matched to velocity
- Workspace min: Full reachable workspace
- Workspace max: Upper bound on arm
- Plan_only = false: plan and execute in one call
- Replan: allow automatic replanning if interrupted.
- Pose_link: Target link for cartesian pose.
- Position tolerance: 5mm, precise to grasp, loose for reliable IK.
- Orientation tolerance: First axis to relax if planning fails.



5.8 Motion Planning and Execution

Each ADL task is its own independent ROS2 node. All three share the same structure: subscribe to /adl_cmd, wait for matching command string, and execute the task sequence by calling helper_moves.py and GetTagPose, recovering on home after a failure. Each task is defined below:

5.8.1 Clear Table (Requirement 2)

Clear up to three household objects – cup, TV remote, or cube – from the table surface. Each object is grasped and placed at a predefined destination location. The task will continue whether or not an earlier object fails.

- **Inputs:** /adl_command='clear_table', /detected_tag_ids, GetTagPose responses for each ID
- **Outputs:** Arm motions, /picked_ids on each successful pick, log messages for debugging, go_home on completion.
- **Success Criterion:** At least 3 objects cleared in >80% of simulation runs; 70% in real hardware trials.

5.8.2 Pick Up Fallen Water Bottle (Requirement 3)

Detect a water bottle lying on its side on the floor, grasp it at floor level, lift it to the table's height, and place in a retrievable location near the user's edge/

- **Inputs:** /adl_command='pick_dropped_bottle', GetTagPose responses for each ID
- **Outputs:** Arm motions, /picked_ids = [0] on successful pick, log messages for debugging, go_home on completion.
- **Success Criterion:** Detect, grasp, lift, and place in >80% of simulation runs; 70% real hardware.

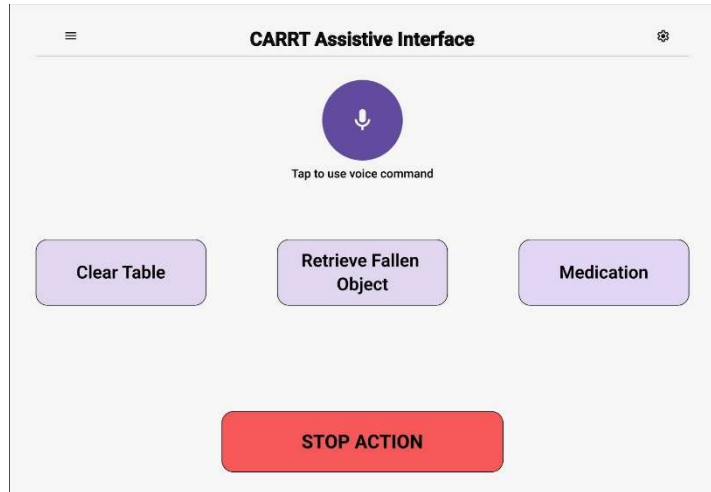
5.8.3 Medication Hand-Off (Requirement 4)

Detects and reads a medication bottle on a table, presents the detected patient's name to the user for confirmation, and hands off the bottle only if the user confirms. If the names do not match, the arm will not move. This is a critical function.

- **Inputs:** /adl_command='give_medication', GetTagPose responses for each ID, user UI response
- **Outputs:** Camera moves and reads label, UI displays detected name and prompts confirmation, arm grasps and delivers only on correct confirmation and UI displays outcome. Arm doesn't move on mismatch.
- **Success Criterion:** Correct match and handoff in 80% of simulation runs. Name mismatch will always abort without motion.

5.9 User Interface

The UI is implemented as a web application (adl_ui/figma_to_ros) served from the ROS2 machine. The user can access it from a tablet browser and it publishes to /adl_command and displays a live status from the executing task node. In the accompanying figure, the current screen interface is depicted.



6. Risk analysis and mitigation

Describe the key risks to project completion and describe what you will do to minimize (or mitigate) these risks. A table can be used here.

The development of the assistive robotic system involves several potential risks that may affect system performance, safety, or project completion. These risks are identified early in the design process so that mitigation strategies can be implemented during development and testing.

Risk	Description	Mitigation Strategy
Vision detection failure	AprilTag markers may not be detected due to lighting conditions or camera angle	Use high-contrast tags, maintain stable lighting conditions, and verify detection reliability in simulation before testing on real hardware.
Motion planning failure	MoveIt may fail to generate a valid trajectory due to collisions, unreachable poses, or planning error	Use collision objects in the planning scene and implement recovery behaviours such as returning to the Home position.

Simulation vs. real-world differences	Behaviour observed in Gazebo simulation may not perfectly match real robot behaviour.	Perform extensive simulation testing and gradually transition to real hardware testing after achieving consistent results.
ROS2 communication failure	ROS2 nodes may fail to communicate properly due to incorrect topic names, message types or node crashes	Use modular node testing and ROS2 debugging tools to verify topic communication and service responses.

7. Design trade-offs

Each trade-off below records a specific design decision where at least one viable alternative was rejected. The selected option is not the only possible solution; it is the option chosen because it best satisfies the project constraints of course schedule, user safety, fixed workspace geometry, accessible task initiation, simulation-before-hardware testing, and repeatable ADL demonstrations.

7.1 Perception method: AprilTag markers selected over learned object detection

Options considered: AprilTag fiducial markers on all manipulable objects and destinations; YOLO or another learned object-detection model.

Selected option: AprilTag-based perception.

Rejected option: YOLO / learned object detection.

Reason for selection: The project requires reliable identification and pose localization of a small, predefined set of household objects in a structured lab workspace. AprilTags provide both object ID and pose estimation without collecting, labeling, training, or validating a custom image dataset. This directly supports the real-time perception requirement while reducing schedule risk.

Accepted trade-off: The system is less flexible than learned object detection. It only works for tagged objects whose markers are visible to the wrist camera, and failures can occur from tag occlusion, poor lighting, bad tag placement, or camera-to-base calibration error. This limitation is accepted because the project assumes a known object set and a controlled workspace.

7.2 Motion interface: direct MoveGroup action client selected over moveit_py

Options considered: moveit_py Python interface; direct use of moveit_msgs through the /move_action MoveGroup action server.

Selected option: Direct MoveGroup action client using moveit_msgs.

Rejected option: moveit_py.

Reason for selection: The direct action-client path was selected because it could be built and executed in the team's ROS 2 Jazzy environment, while moveit_py was not available through the binary installation and could not be reliably built from source during the project timeline. Choosing the lower-level message interface prevented the motion-planning interface from blocking the rest of the ADL implementation.

Accepted trade-off: The direct action-client approach is more verbose and more error-prone than moveit_py. It requires the team to manually construct planning requests, tolerances, workspace bounds, and result checks. This cost is reduced by centralizing the logic inside helper_moves.py instead of duplicating it in each ADL task node.

7.3 Planning strategy: hybrid free-space and Cartesian motion selected over one planning mode

Options considered: free-space planning only; Cartesian motion only; hybrid free-space planning with Cartesian approach and retreat motions.

Selected option: Hybrid motion planning.

Rejected options: Free-space-only planning and Cartesian-only motion.

Reason for selection: Free-space MoveIt planning is appropriate for longer movements around the table, wheelchair, wall, and other collision objects. Cartesian motion is more appropriate for the final grasp approach, lift, and retreat because these steps require predictable tool-frame motion near the object. The hybrid strategy therefore separates gross collision-aware travel from precise local manipulation.

Accepted trade-off: Hybrid planning introduces more state transitions, fallback cases, and failure points than a single planning method. A plan can still fail at the handoff between the free-space pose and the Cartesian approach. This complexity is accepted because free-space-only motion can produce awkward end-effector paths near grasp targets, while Cartesian-only motion is not suitable for longer obstacle-avoiding movement.

7.4 Scene representation: combined static and dynamic planning scene selected over single-source scene geometry

Options considered: static scene only; real-time vision scene only; combined static scene with dynamic AprilTag-based objects.

Selected option: Combined static and dynamic planning scene.

Rejected options: Static-only scene and vision-only scene.

Reason for selection: The table, wheelchair, floor, and wall are known in advance and can be modeled as static collision objects. The manipulated objects change state during tasks and therefore need to be added, updated, or removed dynamically from AprilTag detections. Combining both sources gives MoveIt stable geometry for fixed obstacles while still allowing task objects to be represented as they are detected and picked.

Accepted trade-off: Static geometry must be updated whenever the physical lab setup changes, and dynamic object geometry depends on fresh, correct tag poses. A stale or inaccurate detection can create a wrong collision object or grasp target. This trade-off is accepted because it gives safer collision-aware planning than ignoring fixed obstacles or relying entirely on live vision for every scene element.

7.5 Test environment: simulation-first staged validation selected over hardware-first development

Options considered: physical hardware first; RViz / MoveIt visualization only; staged simulation-first development using RViz, Gazebo, and then physical hardware.

Selected option: Staged simulation-first validation.

Rejected options: Hardware-first development and RViz-only validation.

Reason for selection: Hardware-first development was rejected because early motion-planning, perception, or grasp errors could cause collisions with the table, wheelchair, user region, or robot itself. RViz-only validation was also rejected because it helps verify planning requests but does not sufficiently represent task execution behavior. The chosen staged approach supports the

requirement that ADL tasks be demonstrated in simulation before physical testing and reinforces the 80% simulation-success threshold before hardware trials.

Accepted trade-off: Simulation does not fully capture real gripper contact, object friction, camera noise, calibration error, and hardware timing. Therefore, success in simulation is necessary but not sufficient for final validation. This limitation is accepted because it reduces safety risk and gives the team a repeatable environment for debugging before physical trials.

8. Project plan

- Week 1 – 1/12 – Course Begins
- Week 2 – 1/19 – Establish Company Contact and Requirements/Expectations
 - Lock down the three ADL tasks: (currently)
 - Pick something up from the floor
 - Clear/organize the table
 - Flip a light switch on/off
 - User Stories:
 - “As a daily user, I want a robotic arm that can grab my water bottle from the floor, so that its easily retrievable if knocked over.”
 - “As a daily user, I want a robotic arm that can clear an area of a table, so that it is then available for another use.”
 - “As a daily user, I want a robotic arm that can turn on or off a light, so that I don’t need to ask for help.”
 - Determined QR Recognition for Perception
 - Weekly Status Reports to CARRT
- Development Begins
- Week 3 – 1/26 – Solidify Requirements, Begin UI Planning / QR Code Recognition
 - Determine steps to implement each ADL task, and success criteria.
 - Draft requirements document
 - Begin UI implementation planning
 - Prepare base QR recognition for demo
 - **Status Report / Demo Due 1/30**
- Week 4 – 2/2 – Continue Week 3, Solidify UI and QR Code Architecture
 - Finalize UI Layout
 - Finalize QR code rules and camera integration
 - Start QR detection prototype
 - **Submit Refined Requirements Document 2/6**
- Week 5 – 2/9 – Build System Skeleton
 - Begin integrating QR object detection and pose estimation for tasks.
 - **Status Report / Demo Due 2/13**
- Week 6 – 2/16 – Add Task-Specific Behavior
 - Create support for the 3 ADL tasks in the code.
 - Create light switch detection/interaction logic.
 - Improve QR detection as needed.
 - **Prepare: Design Peer Review Presentation 2/20**
- Week 7 – 2/23 – Review and Refinement
 - Use Peer/Professor feedback to improve design

- ADLs, implementation, etc.
- *Week 8 – 3/2 – Finalize Specification*
 - ✓ *Finalize Design / Logic / UI*
 - ✓ *Document ADLs and component usage.*
 - ✓ ***Status Report / Demo Due 3/6***
 - ✓ ***Submit Specifications Document 3/6***
- Week 9 – 3/9 – Testing and Evaluation Planning
 - Design/Complete Test Plan
 - Determine Metrics for Refinement
 - **Submit Test Plan Document 3/13**
- *Week 10 – 3/16 – Spring Break*
 - *Spring Break*
- Week 11 – 3/23 – Ensure Demo Viability
 - Ensure each task functions properly.
 - Demonstrate each ADL task.
 - Begin optimization.
 - **Demo Due 3/27, Schedule with Professor**
- Week 12 – 3/30 – Optimization Phase Begins
 - Improve design and functionality (consistency)
 - Reduce failure cases.
 - Begin the final data collection.
 - Start drafting the final report.
 - Begin drafting for publication, with collected data.
- Week 13 – 4/6 – Performance Check / Data Analysis
 - Optimize accuracy and reliability.
 - Analyze data for distinguishable patterns.
 - Finish and begin editing of the paper for publication.
 - **Status Report / Demo Due 4/10**
- Week 14 – 4/13 – Final Report Drafting / Figure Creation
 - Finalize report elements
 - Finalize paper for publication.
 - Begin design of other final submission elements.
- Week 15 – 4/20 – Final Draft
 - **Finish Paper for Publishing**
 - **Status Report / Demo Due 4/24**
- Week 16 – 4/27 – Final Submission Due
 - Final Demonstration and Presentation
 - Check over final deliverables
 - Submit complete project
 - **Final Presentation Due 5/1**

9. Traceability to Requirements

1. As a daily user, I want to select ADL tasks with an accessible interface, so that I can control the arm independently.

- a. The system should provide a tablet UI that allows a user to use either voice or buttons to select three ADLs: Clear Table, Pick Up Bottle, Medication Hand-Off
 - b. The UI should not require more than two user actions to start a task (activate listening + one voice command, or one button press).
 - c. The UI should display the current task status with clear text or icons (eg. Idle, Executing, Done, Error, Cancel) and update within 2 seconds of a state change.
- ✓ This requirement is met through Design points [5.9] where UI requirements are directly discussed.
- 2. As a daily user, I want the robotic arm to clear light household objects from a table, so that the surface can be used for another activity.
 - a. [5.5-5.6] and [5.8]: The system should identify at least 3 items located on top of a table's surface, within the Kinova Gen3 arm's workspace.
 - b. [5.7-5.8]: For each identified object, the system should plan and execute a motion to:
 - i. Grasp the item, and:
 - ii. Move the item to its predefined drop location (eg. bin / shelf) within the workspace.
 - c. [5.8]: In at least 80% of simulation runs, the system should clear at least 3 items in one execution of the ADL without collisions with the table, robot arm, or wheelchair.
 - d. [5.8]: During real trials, the system should complete the ADL with at least 70% accuracy in moving all selected items, without collisions.
- ✓ This requirement is met through design points [5.2-5.8], where coding choices determine the creation of the given ADL task, clear_table to interact with nearby household objects.
- 3. As a daily user, I want the robotic arm to pick up a fallen water bottle from the floor, so that I can easily retrieve it if dropped.
 - a. [5.5, 5.8]: The system should detect the pose of the water bottle on the floor, as defined in the Glossary, lying within the workspace of the arm.
 - b. [5.7, 5.8]: The system should grasp the water bottle and lift it to tabletop height about the floor.
 - c. [5.7, 5.8]: The system should then place the bottle at a specified location on the table, such as near the edge.
 - d. [5.8]: In simulation, the ADL should succeed in detecting, picking up, and placing the bottle in at least 80% of trials.
 - e. [5.8]: In real tests, the ADL should succeed in at least 70% of runs without collisions with the floor, table, wheelchair, or robot self-collisions.
- ✓ This requirement is met through design points [5.2-5.8] where coding choices determine the appropriate actions and movement calls for the task pick_dropped_bottle.
- 4. As a daily user, I want the robotic arm to help with medication administration by handing me the correct bottle, so that I don't need outside assistance to do so.

- a. [5.5, 5.8]: The system should detect and identify a medication bottle on a shelf or table, using its QR code, and move to read the label.
 - b. [5.8, 5.9]: The UI should display or announce the detected medication and ask the user to confirm their name.
 - c. [5.7, 5.8]: If the user's name matches the label on the bottle, the system will execute to:
 - i. Grasp the medication bottle, and
 - ii. Move it to a handover location, within the user's reachable area.
 - d. [5.8, 5.9]: If the medication does not have the user's name, the system should not move the arm, display or announce the error, and end the action.
 - e. [5.8]: In simulation, the system should correctly match and hand off medication in 80% of the runs.
- ✓ This requirement is approached by applying a combination of all design points, [5.2-5.8]
5. As a developer, I want the robotic arm to use QR-based perception, so that object and interactable locations are well recognizable.
- a. [5.2, 5.5]: QR markers should be clear and visible at short-range distances.
 - b. [5.5]: The correct target should be identified in at least 80% of perception attempts.
 - c. [5.4, 5.5]: The perception system should operate in real time.
- ✓ This requirement was approached through a combination of design points, to verify correct intended behavior by the camera interface.
6. As a developer, I want to run the operations in simulation first, so that the system behavior is tested safely and repeatedly, and real-world collisions are avoided.
- a. [5.2, 5.4, 5.6]: The system should run in ROS2 and Gazebo for simulation, including environment objects (table, floor, medication bottle, shelf).
 - b. [5.8]: All three ADL tasks should be demonstrable in simulation before real testing is implemented.
 - c. [5.2, 5.6]: The simulation should include any necessary environment objects (table, switch, objects).
- ✓ This requirement is inherently met by our project design allowing simulation in software before deployment to real technology.

10. Traceability to needs and factors

10.1 Public health needs

While not directly stated in the requirements, public health needs are indirectly addressed by the project, where we are hoping to improve disabled user's daily life through supporting them with ADLs. By reducing the need for disabled users to over-exert themselves, or ask for outside assistance, the system could lower risk of strain or injury.

- ✓ Relevant design sections: 5.8.1, 5.8.2, 5.8.3

10.2 Public safety needs

Public safety is driven by requirement (6). This requirement describes the need for simulation and not simply running tests and otherwise on the real equipment. This ensures the safety of developers and future users.

✓ Relevant design sections: 5.6, 5.7, 5.8.3, 6

10.3 Public welfare needs

Public welfare, similarly to public health, is indirectly addressed through project goals of increasing user independence. Through assistance with ADLs, (in requirements (2), (3), and (4)) the robotic arm works to improve the quality of life for disabled individuals.

✓ Relevant design sections: 5.8, 5.9

10.4 Global factors

Global factors are not addressed by the project, as the goal is more localized. The system doesn't involve global effects, since it is only applicable as an assistive robot for those with disabilities.

10.5 Cultural factors

Cultural factors are also not addressed by this project. The robotic arm works to provide basic assistance in daily tasks present over the world and is typically unaffected by a user's culture.

10.6 Social factors

Social factors are addressed through the focus on the goal of independence and improved accessibility. The arm supports users in everyday tasks that they'd often otherwise need assistance for, reducing social dependency and improving user confidence and regular relations in shared spaces. This is directly mentioned in requirement (4), where the goal is a lack of outside assistance, though this can be applied to the other two ADL tasks as well.

✓ Relevant design sections: 5.8, 5.9

10.7 Environmental factors

The robotic arm does not generate significant waste, so the project is only minimally related to environmental factors. It remains limited to local electricity/hardware usage, which are not within the scope of the project.

10.8 Economic factors

Economic factors are indirectly mentioned in Requirement (6). Through the use of simulation software, and through that software being open-source, the cost of development dramatically is reduced. Otherwise, since we were provided with the hardware, the cost was decently low already. Also, while cost analysis lies outside the project's scope, there is still potential in assistive robotic technology to improve long-term caregiving costs.

✓ Relevant sections: 7 (simulation trade-offs), 8

References

- [1] Kinovarobotics, "ros2_kortex: ROS 2 driver for the Gen3 Kinova robot arm," GitHub Repository. [Online]. Available: https://github.com/Kinovarobotics/ros2_kortex (Accessed: Mar. 2026).
- [2] PickNik Robotics, "MoveIt 2 Documentation," 2024. [Online]. Available: <https://moveit.picknik.ai/main/index.html> (Accessed: Mar. 2026).
- [3] E. Olson, "AprilTag: A robust and flexible visual fiducial system," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), Shanghai, China, 2011, pp. 3400–3407. tag36h11 image set: <https://github.com/AprilRobotics/apriltag-imgs>
- [4] Open Robotics, "ROS 2 Jazzy Jalisco Documentation," 2024. [Online]. Available: <https://docs.ros.org/en/jazzy/> (Accessed: Mar. 2026).
- [5] W. Meeussen, "REP-105: Coordinate Frames for Mobile Platforms," ROS Enhancement Proposal 105, 2010. [Online]. Available: <https://www.ros.org/reps/rep-0105.html>
- [6] W. Garage, "REP-103: Standard Units of Measure and Coordinate Conventions," ROS Enhancement Proposal 103, 2010. [Online]. Available: <https://www.ros.org/reps/rep-0103.html>
- [7] Robotiq Inc., "2F-85 and 2F-140 Instruction Manual," Rev. L, 2021. [Online]. Available: <https://robotiq.com/support/2f-85-adaptive-robot-gripper>
- [8] Intel Corporation, "Intel RealSense D415 Datasheet," 2019. [Online]. Available: <https://www.intelrealsense.com/depth-camera-d415/>
- [9] International Organization for Standardization, "ISO 10218-1:2011 — Robots and Robotic Devices — Safety Requirements for Industrial Robots — Part 1: Robots," 2011.
- [10] International Electrotechnical Commission, "IEC 62061:2021 — Safety of Machinery — Functional Safety of Safety-Related Control Systems," 2021.

Test Plan: CARRT Robotic Assistive Arm Project

**Oliver Gaston
Brielle Ashmeade
Allison Moline
Jadon Orr**

Bellini College of AI, Cybersecurity and Computing
University of South Florida
Tampa, FL 33620

Email:

gaston13@usf.edu
ashmeadeb@usf.edu
allison439@usf.edu
jjorr@usf.edu

Version 2.0
March 13, 2026

History of document

- **Version 1.00** (March 10, 2026) – Initial document created
- **Version 2.00** (March 12, 2026) – Body for document completed, to review before submission

Table of Contents

1. Introduction	X
2. Glossary	X
3. Parameters to be tested	X
4. Design of testing	X
5. Test case #1	X
6. Test case #2	X
References	X

1. Introduction

This test plan defines the verification and validation strategy for the CARRT Assistive Robotic Arm system, a Kinova Gen3 robotic arm mounted on a wheelchair that is capable of assisting users with limited upper-limb mobility in completing Activities of Daily Living (ADLs). The three ADL tasks under tests are: (1) Clear Table, (2) Pick Up Fallen Water Bottle, and (3) Medication Hand-Off.

Testing proceeds in two phases: simulation in ROS2 with Gazebo on WSL Ubuntu 24.04, followed by real-world testing on the physical Kinova Gen3 arm. No ADL task with a simulated success rate under 80% may be tested on real hardware (Requirement 6). All testing measures defined here, satisfy a minimum 10-trial requirement.

For the testing purposes of this document, we have selected tasks (1) and (3) to be tested.

2. Glossary

Action Client: ROS2 action instance to send goals and retrieve results.

AprilTag: Marker used for object identification and pose estimation.

End-Effector: end gripper of a robotic arm; the “hand/fingers” of the arm

GetTagPose.srv: Custom ROS2 service returning tag pose in base_link

Home Position: Safe resting configuration programmed into the Kinova Gen3 Arm.

MoveGroup: MoveIt2 action server for planning and execution.

Planning Scene: MoveIt internal representation of obstacles and collision objects.

Tool Frame: End-effector’s reference frame

3. Parameters to be tested

The following table lists the parameters to be tested, their source requirement, how it is measured, and its passing threshold.

Parameter	Requirement	Measurement Method	Pass Threshold
User Interface			
Voice Keyword Detection	Req. 1a	Speak each task trigger phrase; verify /adl_command is published within 3 s.	Both keywords detected in $\geq 90\%$ of utterances
Button Press Task Start	Req. 1b	Press task button; count user actions required before arm begins moving.	≤ 2 actions per task start
UI Status Update Latency	Req. 1c	Record timestamp of state change in log vs. UI display update; compute delta.	Update displayed within 2 s of state change
Clear Table			
Objects Identified on Table	Req 2a	Run vision_stub; check detected_tag_ids includes IDs 2, 3, 4 before task start.	All 3 IDs detected in 100% of trials (stub)
Grasp Success per Object	Req 2b-i	Check gripper joint state at close; verify effort and width per object spec.	Gripper width within 10% of target; effort > 0

Object Moved to Destination	Req. 2b-ii	Verify /picked_ids published after each object; verify scene_from_vision removes object from planning scene.	/picked_ids contains object ID; object removed from scene
Clear Table Success Rate	Req. 2c	Run 10 trials; count trials where all 3 objects cleared without collision.	≥8 of 10 trials fully cleared
Pick Up Dropped Bottle			
Bottle Pose Detected	Req. 3a	Verify GetTagPose(0) returns $z < 0.10$ m and success=true.	Correct floor-level pose returned in 100% of stub trials
Bottle Grasp + Lift Success	Req. 3b	Verify arm reaches floor-level grasp pose and gripper closes; verify retract to $z=0.55$ m succeeds.	Gripper closes and retract completes in ≥80% of trials
Bottle Placed on Table	Req. 3c	After retract, verify arm moves to destination pose (tag 5, near user); verify open_gripper completes.	Destination reached and gripper opens in ≥80% of trials
Pick Bottle Success Rate	Req. 3d	Run 10 trials; count trials where bottle detected, grasped, lifted, and placed correctly.	≥8 of 10 trials succeed end-to-end
Medication Handoff			
Medication Label Read	Req. 4a	Place label bottle in view; verify vision node publishes patient name string before 5 s timeout.	Correct name read in ≥80% of detection attempts
UI Name Confirmation Prompt	Req. 4b	Verify UI displays detected name and prompts user input field after label read.	Prompt displayed in ≤3 s of name detection in 100% of trials
(In)/Correct Name: Cancel / Execute Handoff	Req. 4c-d	Enter name; verify arm grasps bottle and moves to handover pose (tag 5) if matching, or retreats to home and displays UI message.	Handoff completes in ≥80% of matched-name trials, never executes in unmatched
Medication Sim Success Rate	Req. 4e	Run 10 trials with correct name; count end-to-end handoff successes.	≥8 of 10 matched-name trials succeed
Perception and Simulation			
AprilTag ID Accuracy	Req. 5b	Run 10 detection attempts per object; count correct ID reads.	≥80% correct ID per tag in real-hardware phase
Perception Update Rate	Req. 5c	Echo /detected_tag_ids with timestamps; compute publish interval.	Published at ≥5 Hz; inter-message latency ≤200 ms

4. Design of Testing

4.1: Testing Environment — Simulation

All simulation tests are run on WSL Ubuntu 24.04 with ROS2 Jazzy and Gazebo. The following launch sequence must be completed before any test case is executed, without the ADL UI. Source environment variables on each terminal's node:

Environment setup (run in every terminal before the node command):

```
source /opt/ros/jazzy/setup.bash
source ~/workspace/ros2_kortex_ws/install/setup.bash
source install/setup.bash
```

T1 — MoveIt + Fake Hardware:

```
colcon build --symlink-install --packages-select adl_tasks
ros2 launch kinova_gen3_7dof_robotiq_2f_85_moveit_config
robot.launch.py \
    robot_ip:=192.168.0.1 use_fake_hardware:=true
```

T2 — Static Scene:

```
ros2 run adl_tasks scene_static
```

T3 — Vision Stub:

```
ros2 run adl_tasks vision_stub
```

T4 — Scene from Vision:

```
ros2 run adl_tasks scene_from_vision
```

T5 — Task Node:

```
ros2 run adl_tasks clear_table           # Test Case 1
ros2 run adl_tasks pick_dropped_bottle  # Test Case 2
ros2 run adl_tasks give_medication
```

T6 — Trigger command (or use voice/UI button):

```
ros2 topic pub --once /adl_command std_msgs/msg/String "{data:
'clear_table'}"
```

Wait for all nodes to print their own ready log before issuing a command. Confirm that /detected_tag_ids is publishing:

```
ros2 topic echo /detected_tag_ids
```

4.2: Grasp Success — Simulation

Because objects don't visually move in the Gazebo simulator, grasp success is monitored by joint states. A grasp is marked as successful when the following are observed:

- The gripper joint (gen3_robotiq_85_left_knuckle_joint) is within 10% of the target width defined in apriltag_key.py for the object.
- The joint effort reported in /joint_states shows a non-zero resistance value, showing the gripper has contacted the object.

After a successful pick, `scene_from_vision.py` removes the object's collision entry from the MoveIt planning scene. Testers should verify object removal from the planning scene via Rviz or by inspecting the service log for `/apply_planning_scene`.

4.3: Trial Scoring

Each trial is scored as Pass or Fail. A trial *only* passes if every expected output listed in the test case is observed. If any single output is missing or incorrect, the trial Fails. Record results in the Trial Log Sheet as depicted below. The required pass rate for simulation is >80%, and 100% for the medication mismatch test.

Trial	Date	Tester	Pass / Fail	Failure Stage	Notes
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
TOTAL			/ 10		

5. Test case #1 – Clear Table: All 3 Objects

Clear Table: All 3 Objects	
a) System Configuration	
OS / Platform	WSL2 Ubuntu 24.04, ROS2 Jazzy, Gazebo Simulator (Windows 11 Host)
Nodes Running	T1-T4 per Section 4.1, plus: T5: <code>ros2 run adl_tasks clear_table</code> T6: <code>ros2 topic pub --once /adl_command std_msgs/msg/String "{data: 'clear_table'}"</code>
Vision Node	Vision_stub.py – hardcoded poses for a cup, tv remote, and cube
Stub Poses	Cup: (0.45, 0.05, 0.50) Remote: (0.45, -0.10, 0.50) Cube: (0.45, 0.15, 0.50)
Planning Scene	Table, Wheelchair Obstacle, Back Wall, Floor
Arm Start State	Home Position / Straight up
Minimum Trials	10
b) Inputs to Execute	
Step 1	[Launch environment] Complete the T1–T4 launch sequence from Section 4.1. Wait until all four nodes are ready.
Step 2	[Verify vision stub] In a new terminal, run: <i>ros2 topic echo /detected_tag_ids</i> . Confirm the output list includes 2, 3, and 4.
Step 3	[Launch task node] In terminal T5, run: <i>ros2 run adl_tasks clear_table</i> . Wait for log: 'Clear Table Node ready. Waiting for command.'
Step 4	[Verify arm at home] Run: <i>ros2 topic echo /joint_states --once</i> . Confirm arm is at its home configuration (all joint positions at home values).
Step 5	[Issue command] In terminal T6, run: <i>ros2 topic pub --once /adl_command std_msgs/msg/String "{data: 'clear_table'}"</i>
Step 6	[Observe task log] In the clear_table terminal, watch for: detected object IDs logged, ' <i>Clearing object [name]...</i> ' for each of the 3 objects, and ' <i>cleared successfully</i> ' per object.
Step 7	[Verify /picked_ids] After each grasp, run: <i>ros2 topic echo /picked_ids --once</i> . Confirm the current object's ID is published.
Step 8	[Verify scene update] In RViz or scene_from_vision terminal, confirm the object's collision entry is removed after each pick.
Step 9	[Verify arm return] After all objects are processed, confirm the arm returns to home position in the joint_states output.
Step 10	[Reset and repeat] Reset the planning scene (relaunch T2–T4) and repeat from Step 4 for a total of 10 trials.
c) Expected Outputted Results	
/detected_tag_ids	List includes [2, 3, 4] before arm moves. Published at ≥ 10 Hz.
Task Log – Detection	clear_table terminal logs: 'Detected 3 objects to clear (IDs): [2, 3, 4]'
Task Log – Per Object	For each ID: 'Clearing object [Cup/TV Remote/Cube] (ID [2/3/4])...' then '[name] (ID [X]) cleared successfully.'
Gripper Joint – Per Object	gen3_robotiq_85_left_knuckle_joint position $\approx$ OBJ_WIDTH $\pm$ 10%; effort field non-zero

/picked_ids	Publishes [2], then [3], then [4] in sequence after each grasp
Planning Scene	Each object's collision entry removed in RViz/scene_from_vision log after its pick
Arm Return	Arm returns to home position; joint_states matches home configuration
No Collision	No MoveIt collision error message in any terminal output
Final Log	clear_table terminal logs: 'Clear table task completed. Cleared 3/3 objects.'
Pass Criterion	All expected results observed in a single trial = Pass. Required: ≥ 8 of 10 trials Pass.

6. Test case #2 — Medication Hand-Off: Names Mismatch

Medication Hand-Off: Mismatched Names, Arm Aborts	
a) System Configuration	
OS / Platform	WSL2 Ubuntu 24.04, ROS2 Jazzy, Gazebo Simulator (Windows 11 Host)
Nodes Running	T1-T4 per Section 4.1, plus: T5: <code>ros2 run adl_tasks clear_table</code> T6: <code>ros2 topic pub --once /adl_command std_msgs/msg/String "{data: 'clear_table'}"</code>
Vision Node	Vision_stub.py – hardcoded poses for a cup, tv remote, and cube
Stub Poses	Cup: (0.45, 0.05, 0.50) Remote: (0.45, -0.10, 0.50) Cube: (0.45, 0.15, 0.50)
Planning Scene	Table, Wheelchair Obstacle, Back Wall, Floor
Arm Start State	Home Position / Straight up
Minimum Trials	10
Medication Name	JOHN DOE
b) Inputs to Execute	
Step 1	[Launch environment] Complete the T1–T4 launch sequence from Section 4.1. Wait until all four nodes are ready.
Step 2	[Launch task node] In T5: <i>ros2 run adl_tasks give_medication</i> . Wait for log: 'Medication Node ready. Waiting for command.'
Step 3	[Open UI] <i>Open the browser UI at <code>http://<WSL-IP>:5000</code>. Confirm the interface loads and shows status 'Idle'.</i>
Step 4	[Verify arm at home] Run: <i>ros2 topic echo /joint_states --once</i> . Confirm home position.
Step 5	[Issue command] Press the 'Medication' button in the UI, OR run: <i>ros2 topic pub --once /adl_command std_msgs/msg/String "{data: 'give_medication'}"</i>
Step 6	[Observe label read] In the give_medication terminal, watch for the log indicating the vision node has returned the patient name from the label (e.g., 'Detected patient name: John Doe'). The UI should display the detected name.
Step 7	[Enter wrong name] When the UI displays the name confirmation prompt, type a name that does NOT match the label. Confirm and submit.
Step 8	[Observe arm — must not move] Watch the robot in RViz/Gazebo and monitor /joint_states. The arm must remain stationary at home position.

	This is the critical observation.
Step 9	[Verify UI error] Confirm the UI displays a mismatch error message (e.g., 'Name does not match. Task aborted.').
Step 10	[Verify no /picked_ids] Run: <i>ros2 topic echo /picked_ids</i> . Confirm no message is published during this trial.
Step 11	[Reset and repeat] Refresh UI status to 'Idle'. Repeat from Step 4 for a total of 10 trials. Use a different wrong name each trial to test varied inputs.
c) Expected Outputted Results	
Label Read	give_medication terminal logs patient name read from bottle before timeout.
UI Name Prompt	UI displays the detected name and shows a text input field asking the user to confirm their name. This occurs within 3 s of label read.
Mismatch Detection	UI detects mismatch between entered name and label name (case-insensitive comparison)
Arm Motion	ZERO arm motion. /joint_states shows no change from home position at any point after the command. This must hold in 10/10 trials.
UI Error Message	UI displays a clear error or abort message, e.g., 'Name does not match. Task aborted.' Status updates to 'Idle' within 2 s.
Task Log	give_medication terminal logs: 'Name mismatch. Arm will not move.' (or equivalent). No motion plan is sent.
Pass Criterion	Arm does not move AND UI shows error AND /picked_ids is empty = Pass. Required: 10/10 trials Pass.

7. Traceability Matrix

The table below maps the requirements and specifications to the test cases discussed in this plan. Items not covered by the two cases are marked with additional needed test methods:

Requirement / Spec Item	5.5 Perception	5.7 Motion	5.8 Tasks	Test Coverage
Requirement 1 — Accessible UI				
Req. 1a Voice + button input			✓	Tested via TC-1 button trigger and TC-2 voice pathway [informal]
Req. 1b ≤ 2 actions to start task			✓	TC-1 Step 5: single topic publish / one button press
Req. 1c UI status ≤ 2 s update			✓	Observed during TC-1 execution; no automated timer
Requirement 2 — Clear Table				
Req. 2a Identify ≥ 3 table items	✓		✓	TC-1: detected <code>tag_ids</code> checked for IDs 2, 3, 4 before step 6
Req. 2b-i Grasp each item		✓	✓	TC-1 Step 8: joint state effort and width verified per object
Req. 2b-ii Move to drop location		✓	✓	TC-1 Step 9: <code>/picked_ids</code> and scene update verified
Req. 2c $\geq 80\%$ sim success rate	✓	✓	✓	TC-1: 10 trials, ≥ 8 must pass all criteria
Requirement 3 — Pick Bottle				
Req. 3a Detect floor pose	✓		✓	Not primary TC; verified informally via <code>GetTagPose</code> check
Req. 3b Grasp and lift		✓	✓	Not primary TC; exercise during integration testing
Req. 3c Place on table		✓	✓	Not primary TC; exercise during integration testing
Req. 3d $\geq 80\%$ sim success	✓	✓	✓	Not primary TC; 10-trial run recommended before hardware
Requirement 4 — Medication Hand-Off				
Req. 4a Detect + read label	✓		✓	TC-2 Step 6: vision node publishes patient name string
Req. 4b UI confirm name			✓	TC-2 Step 7: UI prompts user name input
Req. 4c-i Grasp on match		✓	✓	TC-2 Step 7: not exercised (wrong name supplied)
Req. 4c-ii Move to handover		✓	✓	TC-2 Step 7: not exercised (wrong name)
Req. 4d Abort on mismatch (CRITICAL)		✓	✓	TC-2 primary focus: arm must not move on mismatch
Req. 4e $\geq 80\%$ sim success	✓	✓	✓	Matched-name variant recommended as additional 10-trial run
Requirement 5 — QR Perception				

Requirement / Spec Item	5.5 Perception	5.7 Motion	5.8 Tasks	Test Coverage
Req. 5a Tags visible at range	✓			Checked during TC-1 setup: stub always-visible baseline
Req. 5b $\geq 80\%$ ID accuracy	✓			Real-hardware phase; repeat GetTagPose 10 $\times$ per tag
Req. 5c Real-time ≥ 5 Hz	✓			Check topic rate: ros2 topic hz /detected_tag_ids
Requirement 6 — Simulation First				
Req. 6a ROS2 + Gazebo env		✓		TC-1 and TC-2 both run on fake hardware in Gazebo
Req. 6b All ADLs in sim			✓	TC-1 (clear table) and TC-2 (medication) both in sim
Req. 6c Env objects in sim	✓	✓		scene static loaded in TC-1 and TC-2 setup (Steps 1–4)

References

- [1] Kinovarobotics, 'ros2_kortex: ROS 2 driver for the Gen3 Kinova robot arm,' GitHub. Available: https://github.com/Kinovarobotics/ros2_kortex
- [2] PickNik Robotics, 'MoveIt 2 Documentation,' 2024. Available: <https://moveit.picknik.ai/main/index.html>
- [3] E. Olson, 'AprilTag: A robust and flexible visual fiducial system,' Proc. IEEE ICRA, 2011.
- [4] Open Robotics, 'ROS 2 Jazzy Jalisco Documentation,' 2024. Available: <https://docs.ros.org/en/jazzy/>
- [5] Robotiq Inc., '2F-85 and 2F-140 Instruction Manual,' Rev. L, 2021.
- [6] CARRT Robotic Assistive Arm Project — Specification Document v3.0, March 2026.
- [7] CARRT Robotic Assistive Arm Project — Requirements Document v2.0, February 2026.

Design Review: CARRT Assistive Robotic Arm

**Oliver Gaston, Jadon Orr,
Brielle Ashmeade, Allison Moline**
Department of Computer Science and Engineering
University of South Florida
Tampa, FL 33620
gaston13@usf.edu, jjorr@usf.edu
ashmeadeb@usf.edu, allison439@usf.edu

Acknowledgments

- **Thank you, for support and guidance, to:**
 - Dr. Alqasemi, Leading Professor of CARRT Labs
 - Dr. Ken Christensen, Professor of CSE Course
 - Sohrab Moayer, Graduate Student

Agenda

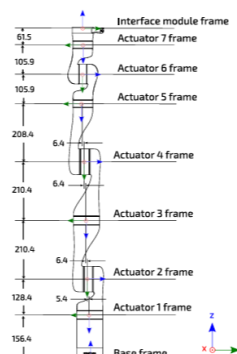
- Background
- Problem
- Requirements
- Design
- Constraints
- Applicable standards
- Risk analysis and mitigation
- Project plan

Background

- Approximately 2.5 million people live with moderate to severe mobility impairments of their arm or hands.
- The CARRT Lab provided our team with access to the Kinova Gen3 robotic arm.
- The ARM is lightweight, programmable and features 7 DoF.



KINOVA GEN 3 ROBOTIC ARM
<https://www.kinovarobotics.com/product/gen3-robots>



ARM'S 7 DoF
<https://www.kinovarobotics.com/uploads/User-Guide-Gen3-R07.pdf>

Problem

- **Problem Statement:**

- Implement three unique ADLs that can be automated by a Kinova Gen3 robotic arm mounted on their wheelchair, specifically for a user with a physical disability affecting upper-limb mobility.
- Specifically, create a system that can:
 - Clear household objects from table
 - Retrieve a fallen water bottle within range
 - Support medication administration

Requirements

1. Accessible Interface:

- Touch controls
- Voice controls

2. Clear Table:

- Identify at least 3 objects on table surface
- Determine and execute motion based on object

3. Pick Water Bottle From Floor:

- Detect position of water bottle
- Grasp water bottle and lift to tabletop height
- Put onto designated area

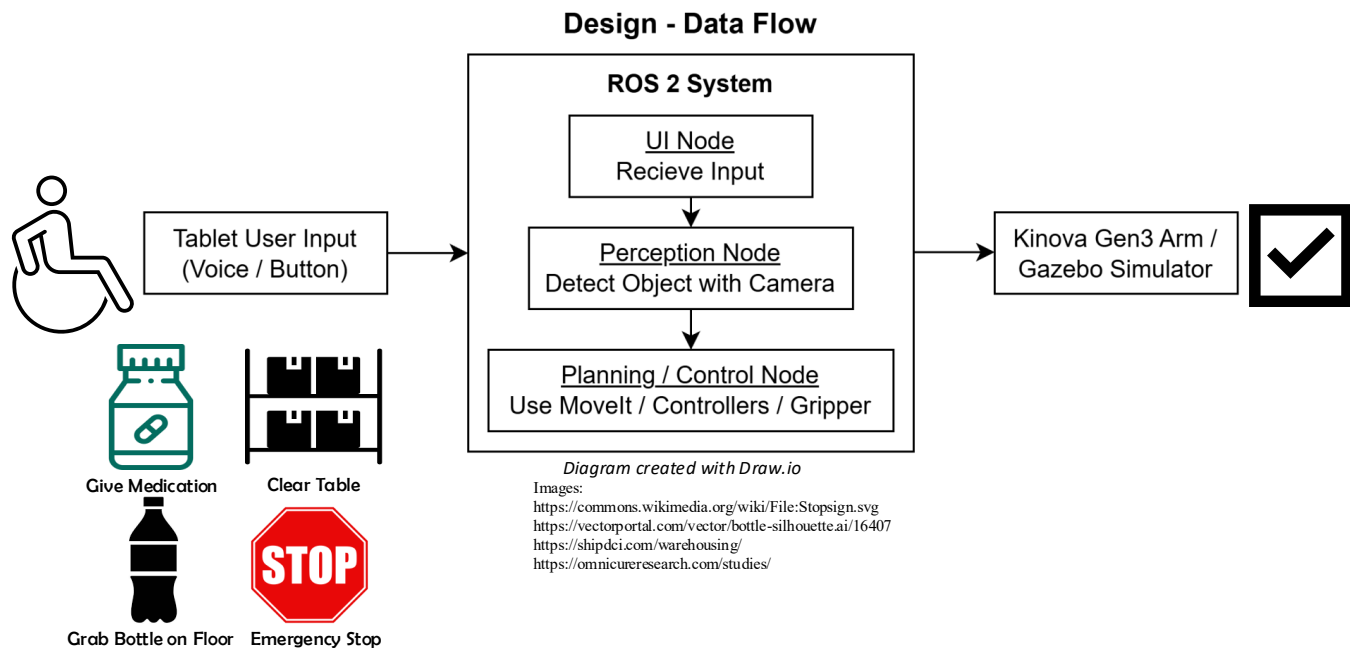
4. Medication Administration:

- Detect medication bottle using QR code
- Grasp medication bottle, move it to a user's reachable area

5. QR-Based Perception

6. Operations Run in Simulator First

Design – High Level Architecture

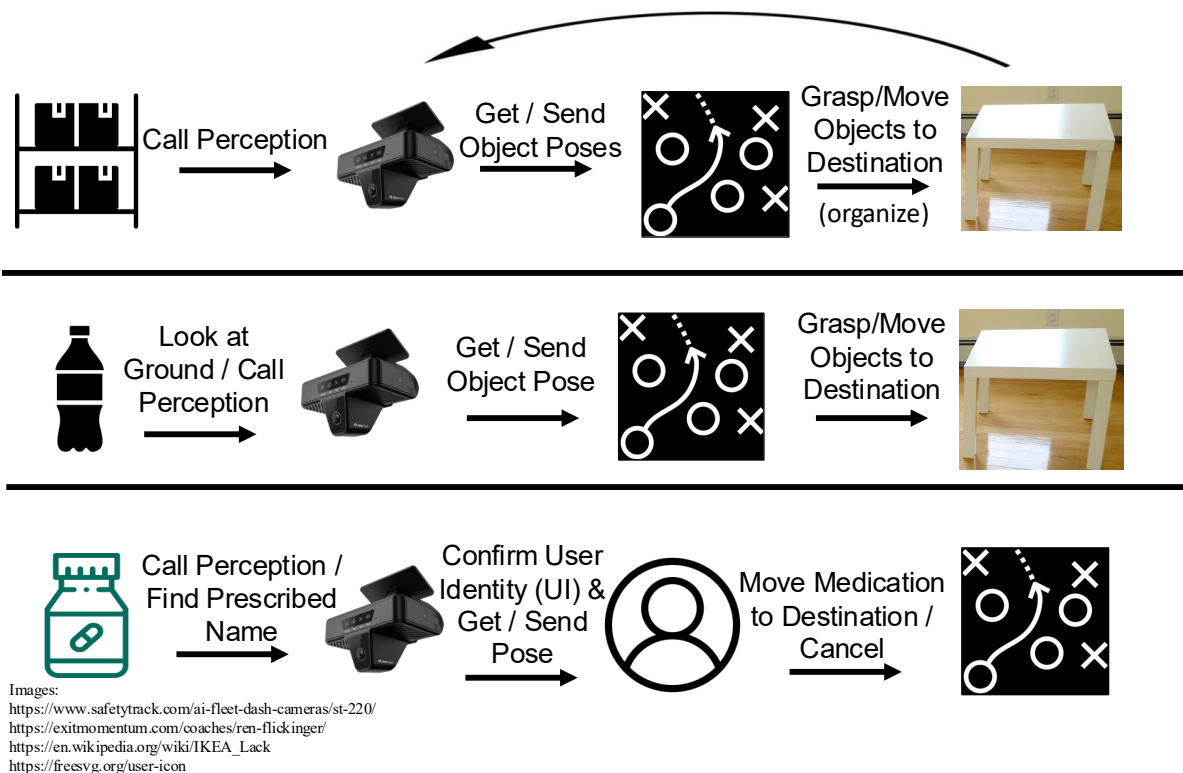


7 of 12

Design Review for CARRT Project
Feb 20, 2026



Design – Data Flow for ADLs



8 of 12

Design Review for CARRT Project
Feb 20, 2026



Constraints

- **User**
 - UI must be accessible for persons with disabilities
- **Hardware & Environment**
 - One Arm, Dual Gripper
 - Lab Environment
- **Perception**
 - Simple camera / QR interface
 - Good lighting conditions
- **Object Size**
 - Graspable: 5-15cm width
 - Weight: 1-2kg



Images:
<https://freesvg.org/skotan-no-sign-x>
https://www.aastateofplay.com/media/catalog/product/P/Plastic_Bowling_Pin_Set_with_5_Pound_Rubber_Bowling_Ball_2.jpg
https://www.flaticon.com/free-icon/approved_7211142
<https://www.mahlwerk.de/kaffebecher-porzellan-bunt-to-go-orange-400-ml>



Applicable standards

- **Robot Safety (Human-Robot Interaction)**
 - ISO 10218-1/2 (2025/2011)
 - ISO/TS 15066 (2016)
 - International Organization for Standardization (ISO)
- **Accessibility / UI**
 - W3C/WCAG Accessibility Guidelines
 - Web Accessibility Initiative
- **Software / ROS2**
 - Naming conventions
 - Simulation standards
 - PEP 8 Python Style



Risk analysis and mitigation

- **Risks**

- Software Crash
- Sensory Misread
- Logic bugs

- **Mitigation**

- Auto restart and handle errors.
- Verify sensor data before acting to avoid mistakes.
- Test the system thoroughly before real-world use.

Project plan

- **Key Milestones**

- Design Review (2/20)
- Demo Viability Check (3/27)
- Final Demonstration (5/1)

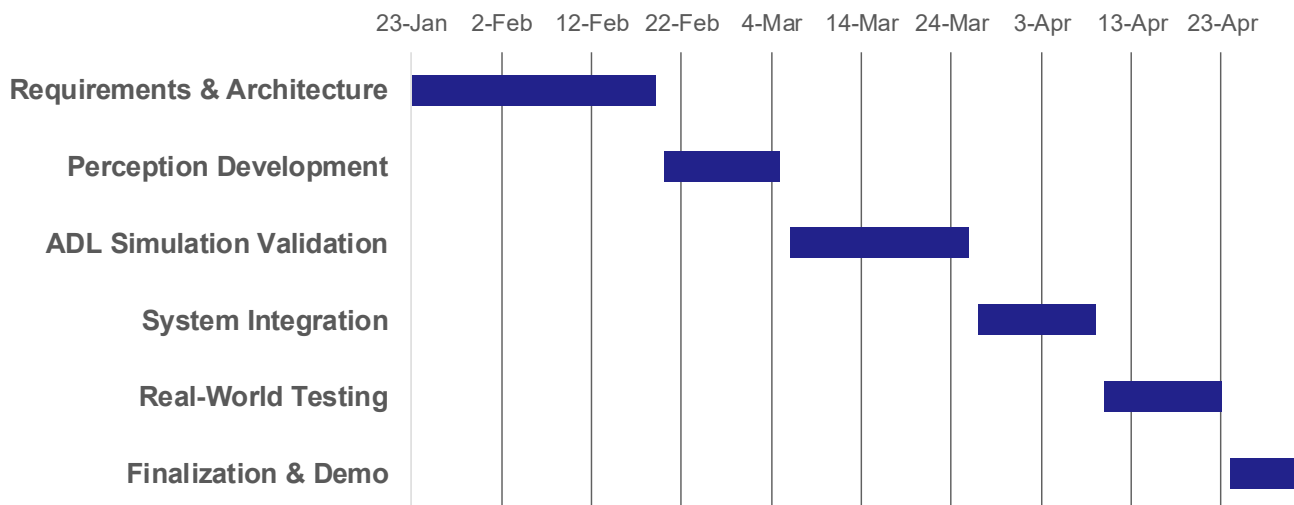


Diagram created using Excel

CARRT Assistive Robotic Arm

**Oliver Gaston, Jadon Orr,
Brielle Ashmeade, Allison Moline**

Department of Computer Science and Engineering
University of South Florida Tampa, FL 33620
gaston13@usf.edu, jjorr@usf.edu
ashmeadeb@usf.edu, allison439@usf.edu

Acknowledgments

- **Thank you, for support and guidance, to:**
 - Dr. Alqasemi, Leading Professor of CARRT Lab
 - Dr. Ken Christensen, Professor of CSE Course
 - Sohrab Moayer, Graduate Student

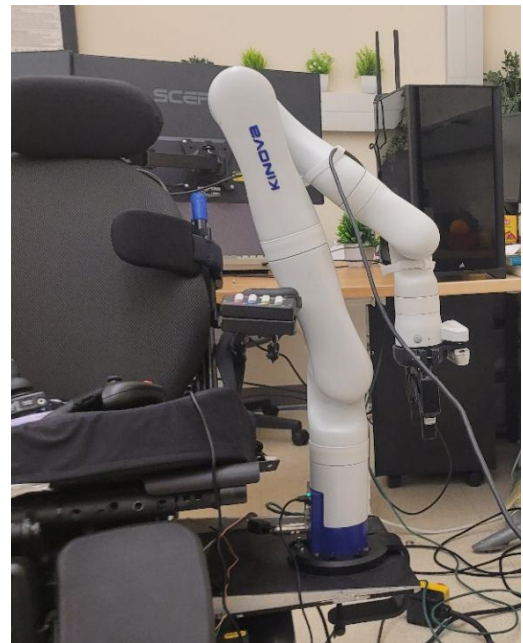
Additional thank you to other **CARRT lab students** and **CSE Project students** consulted for advice on project planning and execution.

Agenda

- **Background**
- **Problem**
- **Requirements**
- **Demo**
- **Customer statement**
- **Design**
- **Constraints**
- **Applicable standards**
- **Trade-offs made**

Background

- Approximately 2.5 million people live with moderate to severe mobility impairments of their arms or hands.
- Wheelchair-mounted robotic arms can assist with Activities of Daily Living (ADLs), but require perception, motion planning, and user control.
- The CARRT Lab provided access to a wheelchair-mounted Kinova Gen3 7-DoF robotic arm and a structured lab workspace.
- Our project created an integrated prototype of an assistive system, not a deployable product.



*Wheelchair-mounted Kinova Gen3 7-DoF
Robotiq Arm from CARRT Lab*

Problem

• Problem Statement:

- Implement three unique ADLs that can be automated by a Kinova Gen3 robotic arm mounted on their wheelchair, specifically for a user with a physical disability affecting upper-limb mobility.
- Specifically, create a system that can:
 - Clear household objects from table
 - Retrieve a fallen water bottle within range
 - Support medication administration

Final Notes: the project produced an integrated prototype but did not reach reliable end-to-end execution.

Requirements

- 1. Accessible Interface: touch and voice commands**
- 2. Clear Table: 70/153 total success/starts (13.0%)**
 - Cup, remote, and cube workflows
 - **Best-Day task success rate:** 8/20 command starts, **40.0%**
- 3. Pick Dropped Water Bottle: incomplete, 100% simulation**
 - Workflow *implemented in software*
 - *Failed to meet constraints:* physical floor-workspace tightness
- 4. Medication Handoff: 13/80 total success/starts (16.3%)**
 - Bottle workflow and name-confirmation logic implemented.
 - **Best-Day task success rate:** 3/13 command starts, **23.1%**
- 5. QR-Based Perception**
 - Tag detection and pose service functional.
 - Offsets/calibration remained source of problems.
- 6. Operations Run in Simulator First**
 - Simulation and fake-hardware paths **used**.
 - Reliability **failed to meet original 70-80%** real-world task success rate goal.

Demo

- **Limited Functionality Overall, Conservative Results**

- In this demo we will show the system's functionality and performance of each ADL Task

- *Please Enjoy!*

- Final success rate did not meet original target within the course timeline, but progress was notable.



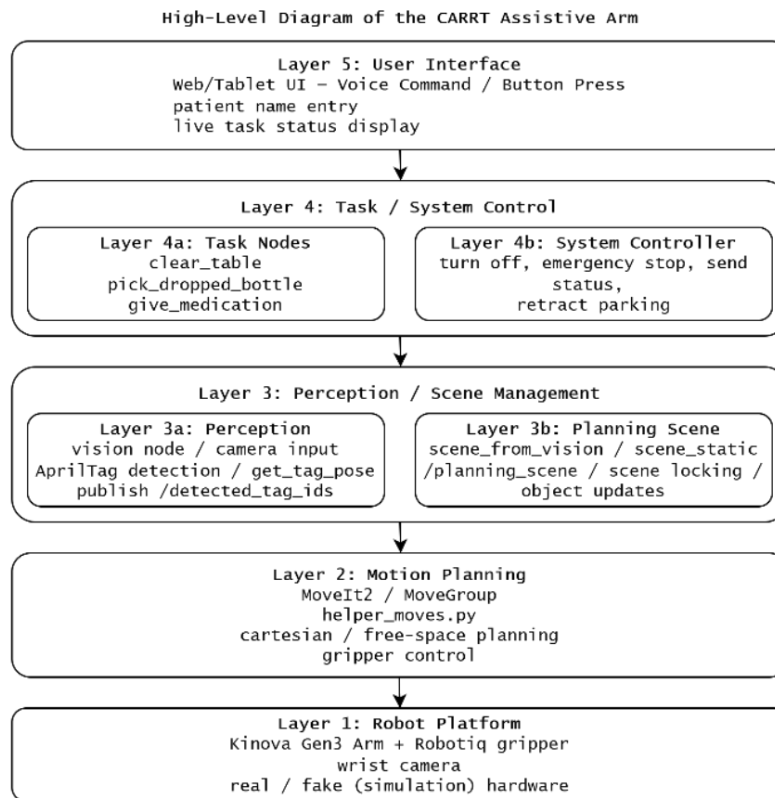
AprilTagged Objects and drop locations

Customer Statement

- **From Dr. Alqasemi:**

"Thank you for presenting a demo of the project. Although the work is not completed as intended, the progress is satisfactory, and the amount of learning you made is notable. Your resilience to address many of the challenges when implementing your work to the hardware is great. It was a pleasure having you in the lab throughout the semester, and I wish you all the best of luck in your life endeavors."

Design - High Level Architecture

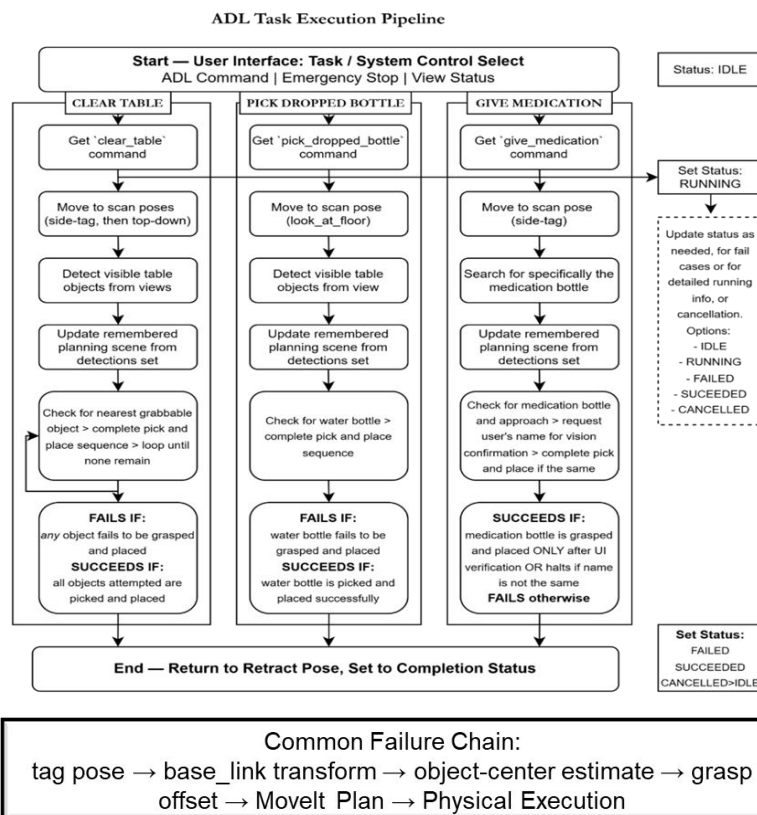


9 of 13

CARRT Robotic Assistive Arm Project
May 1, 2026



Design - Data Flow for ADLs



10 of 13

CARRT Robotic Assistive Arm Project
May 1, 2026



Constraints

- **User**

- UI must be accessible through large buttons and voice command.

- **Hardware & Environment**

- One Kinova Gen3 arm with Robotiq 2F-85 Gripper
- Fixed lab workspace, wheelchair mounted, table, floor, shelf / bin / handoff regions.
- Small workspace, limited grasp and placement options

- **Perception**

- AprilTags must be visible from scan poses
- Expected stable lighting and wrist-camera cable connection
- Calibration to base_link must remain accurate

- **Object Size**

- Small, rigid objects, about 5-15 cm and under 1kg

- **Schedule**

- Course timeline limited repeated physical trials and final tuning

Applicable standards

- **Robot Safety (Human-Robot Interaction)**

- ISO 10218-1/2: industrial robot safety
- ISO/TS 15066: Collaborative robot safety
- IEC 62061: safety-related control system guidance

- **ROS 2 / Coordinate Conventions**

- REP-103: standard units and coordinate conventions
- REP-105: common mobile coordinate frames

- **Accessibility / UI**

- W3C/WCAG Accessibility Guidance for readable, operable interfaces

- **Software Quality**

- ROS 2 package conventions, sim-first development, PEP 8 Python style.

Trade offs

- **Apriltags vs AI object recognition**
 - AprilTags
 - Simpler IDs, less flexible and highly calibration-dependent.
- **Predefined environment vs Dynamic environment**
 - Static Scene + Dynamic Tagged Objects
 - More repeatable testing, less robust to layout changes
- **Free-space Planning vs Hybrid**
 - Fixed scan poses + MoveIt + Cartesian approaches
 - Better repeatability, more tuning points
- **Scope Trade-Off**
 - Three ADL Workflows
 - Matched requirements, but reduced time to make any one workflow fully reliable.

Press Release

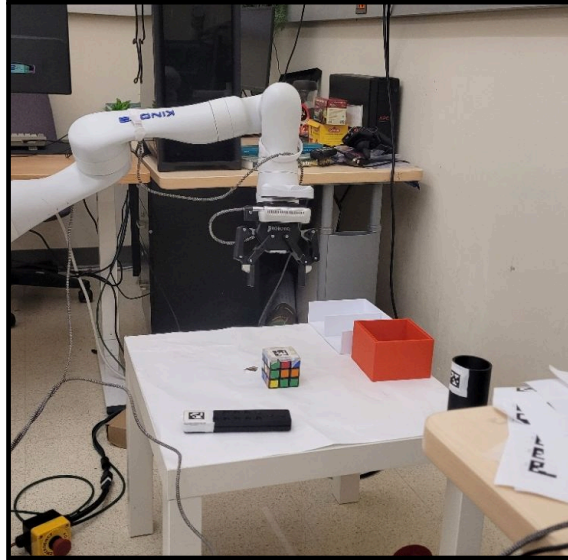


Image: a Wheelchair-mounted Kinova Gen3 robotic arm prototype, developed for the CARRT lab assistive robotic arm project.

CARRT Assistive Robotic Arm Prototype Supports Independent Daily Living

(Tampa, Florida, May 1, 2026) – A University of South Florida senior project team has developed a promising assistive robotic arm prototype designed to help wheelchair users with limited upper-limb mobility complete everyday tasks with greater independence.

The CARRT Assistive Robotic Arm project was created by Oliver Gaston, Jadon Orr, Brielle Ashmeade, and Allison Moline, during the 2026 CIS 4910 CSE Project course, collaborating closely with USF's CARRT Lab. The system uses a Kinova Gen3 Robotic Arm, ROS 2 software, MoveIt 2 motion planning, AprilTag-based perception and an accessible tablet UI, with button or voice-command available.

The prototype was designed to complete three Activities of Daily Living: clearing objects from a table, retrieving a dropped water bottle, and supporting medication hand-off with user-name confirmation. These were selected to address common daily challenges faced by those with limited mobility.

“This project helped instill the value in careful engineering decisions and pipelining, to best support later success,” Said team member Oliver Gaston. “Our goal was not just to move a robotic arm, but to build a system that can connect accessibility, perception, motion planning, and safety into one system.

Although further testing and calibration are required for real-world deployment, the project provides a foundation for future assistive robotics development at USF.

CARRT Assistive Robotic Arm

Oliver Gaston, Allison Moline, Jadon Orr, Brielle Ashmeade



Background

Individuals with reduced upper-limb mobility may require assistance to perform **Activities of Daily Living (ADLs)**, including tabletop organization, dropped-object retrieval, and medication handling. These limitations can reduce independence and make daily tasks more difficult. **Assistive robotic systems** provide a promising solution by enabling users to interact with their environment through automated manipulation and control.

Problem

Implement three unique ADLs that can be automated by a Kinova Gen3 robotic arm mounted on their wheelchair, specifically for a user with a physical disability affecting upper-limb mobility.

Requirements

1. Have an accessible user interface with voice input
2. Clear Table: recognize object using apriltag, put them in a designated spot.
3. Pick Dropped Water-bottle: recognize water bottle on the floor and pick it up
4. Medication Administration: determine if the current user should receive the medication

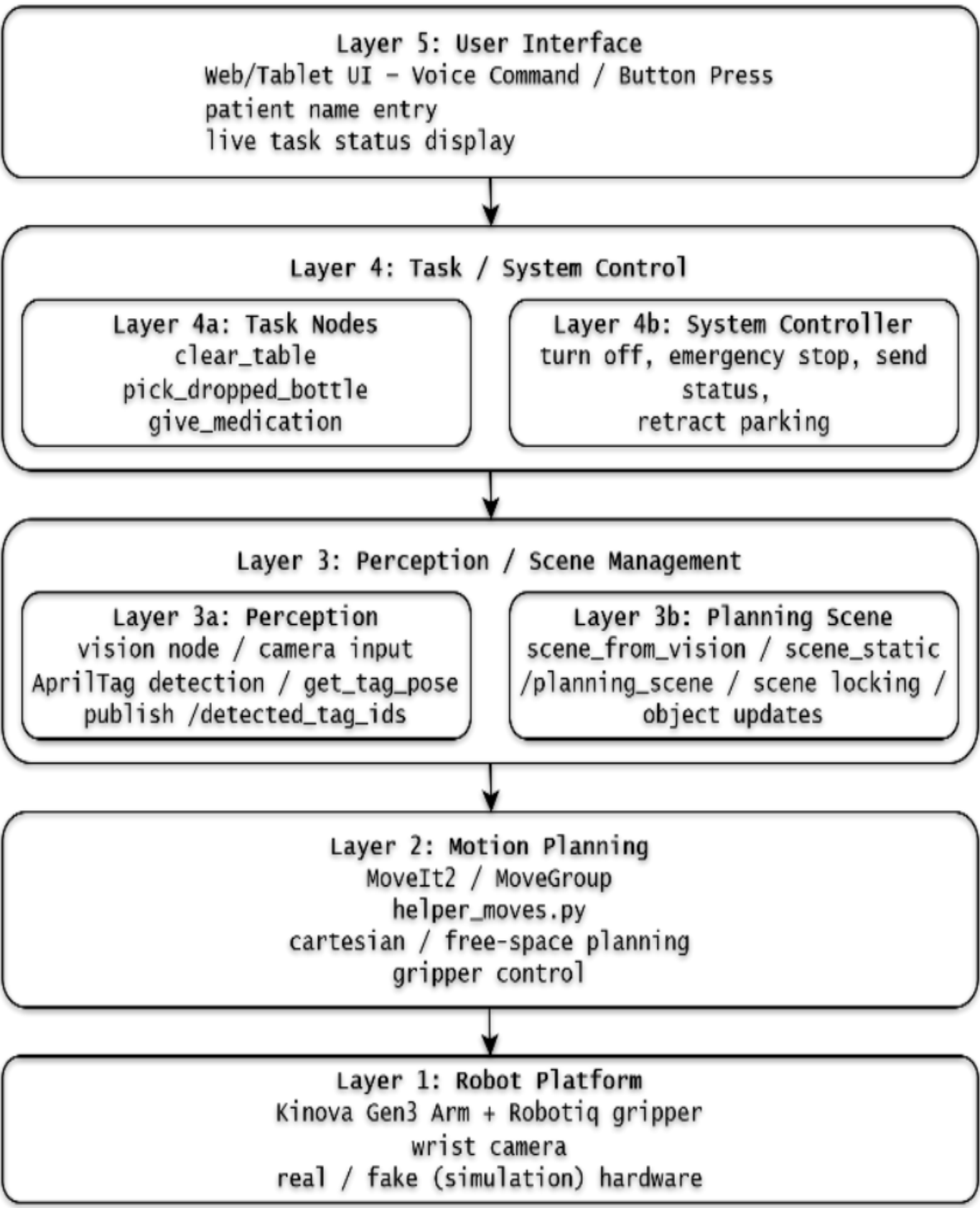
Acknowledgements

We would like to thank the USF CARRT Lab, and specifically Dr. Alqasemi and Sohrab Moayer for advice and guidance during the development of the system. Without your advice and expertise, the project would not have been possible.

Design

The system uses a **7-DoF Kinova Gen3 robotic arm** and **Robotiq 2F-85 gripper**, mounted to a wheelchair and integrates **ROS 2** middleware, **MoveIt 2** motion planning, and **AprilTag-based perception** to detect, localize and manipulate **household objects** within the robot's workspace.

High-Level Architecture Design Layers

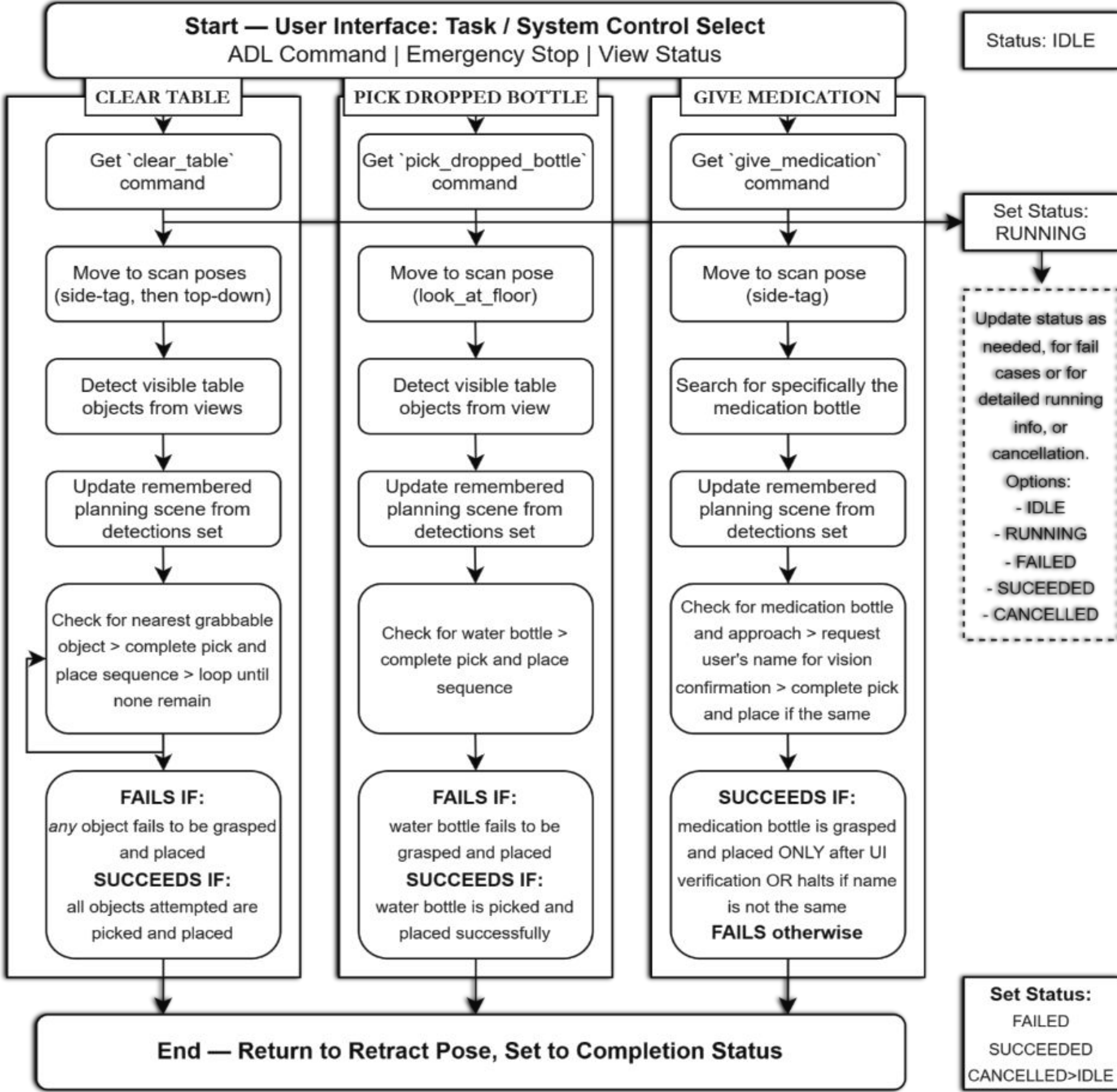


Custom ADL User Interface Display



Simple Custom Flask Web UI with Voice and Button Control, and status display

ADL-Task-Level Execution Pipelines



Constraints / Applicable Standards

- Constraints
- Limited course timeline for development
 - Real-arm testing delayed by simulation phase
 - Fixed lab workspace: table, floors, wheelchair, wall zones
 - Tagged objects only: visible AprilTags
 - Real-time accessible UI

- Standards
- **ROS REP-103 / 105:** units for robot measurements / standard orientation frames
 - **ISO 10218-1:** robot safety, collision-aware motion, speed limits
 - **IEC 62061:** safe-stop and fault-handling guidance

Status Report for CARRT GROUP for Week # 3

What were the goals for this two-week period?

- 1) Select and document three ADL tasks to perform with the Genova Kin 3 ARM by the end of week 3.
- 2) Complete the installation of the Robotic Operating System (ROS) and verify its functionality by running sample publisher-subscriber nodes.
- 3) Complete the installation of Gazebo (robotic simulator) and verify the Genova Kin 3 ARM's functionality by running at least one simulation of the selected ADL tasks.

What goals were accomplished in this two-week period?

- 1) Our first goal was accomplished. We decided on three ADL tasks to perform with the Genova Kin 3 ARM:
 - a) Clearing a Table
 - b) Picking up an object from the ground
 - c) Flipping a light switch on and off
- 2) Our second goal was accomplished as we were able to install ROS and run the sample publisher and subscriber nodes using the Ubuntu Operating System and VS Code.
- 3) Our third goal was not fully accomplished, as we were unable to get the Genova Kin 3 ARM to move in the Gazebo Simulator. We did successfully model the arm in Gazebo, and were able to view and manipulate the workspace after a

Reflect critically on any goals not accomplished.

< Explain why you did not complete any missed goals and what you plan to do moving forward with respect to these goals. >

- 1) Our team missed our third goal because of unfamiliarity with the Gazebo Simulator. Moving forward, we will focus most of our efforts on understanding this simulator so that we can successfully test the three ADL tasks before we start programming the ARM.

What are the goals for next two weeks?

< Enumerate your goals for next two weeks. Be sure that your goals are measurable and connected to your requirements. >

- 1) Successfully test at least two of the three selected ADL tasks using the Genova Kin 3 ARM in the Gazebo simulator.
- 2) Begin developing the user interface for the Genova Kin 3 ARM and determine a method to project it onto an iPad for interaction.

How many hours were spent on each goal noted above in the past two weeks?

< For each person on the team list the goals they worked on including completed and non-completed goals>

Team Member	Goal	Completed?	Hours Spent
Allison	1	YES	15 mins
Oliver	1	YES	15 mins
Brielle	1	YES	15 mins
Jadon	1	YES	15 mins
Allison	2	YES	2 hours
Oliver	2	YES	2 hours
Brielle	2	YES	2 hours
Jadon	2	YES	3 hours
Allison	3	NO	0 hours
Oliver	3	NO	3 hours
Brielle	3	NO	0 hours
Jadon	3	NO	0 hours

Status Report for CARRT GROUP for Week # 4

What were the goals for this one-week period?

- 1) Solidify the requirements of the project, as defined by class assignments into a single PDF file.
- 2) Begin implementation of the ADL tasks in ROS2 and Gazebo, using written Python files.
- 3) Continue implementation of the UI and QR recognition, narrowing down specific QR codes used, and how object recognition will be affected by each type of object.

What goals were accomplished in this one-week period?

- 1) Our first goal was successfully met, with the Requirements document provided alongside this report. We have detailed the goal, main important concepts, assumptions, and overall requirements of the project, when completed successfully.
- 2) Our second goal was accomplished, though with less progress than preferred. We have set up the environment for coding, however due to the kortex_api library not wanting to be accessed, we had to determine the proper steps to find and utilize that package.
- 3) Our third goal was not fully accomplished, as we have only recently been able to find and determine the plan to using the QR codes for object recognition with OpenCV and AprilTag, as suggested. Instead, we have veered towards determining exactly the types of objects, enumerated, that will be used and coded in with their own QR codes, as well as QR code placement.
 - a. Object
 - i. Water Bottle: task 1, 2
 - ii. Block: task 2
 - iii. Light Switch (3-D printed/bought): task 3
 - iv. Pitcher
 - b. Environment
 - i. Table
 - ii. Floor
 - iii. Attached Chair (to avoid collisions)
 - iv. Robot Base QR code? (possibly helpful)

Reflect critically on any goals not accomplished.

< Explain why you did not complete any missed goals and what you plan to do moving forward with respect to these goals. >

- 1) Our team struggled with goal two, as kortex_api refused to install on the main terminal that simulation will be occurring. We were forced to troubleshoot to determine the cause, an outsourced link that was incorrectly written in their install packages. This delayed simulation and code writing, as packages weren't being correctly added for use.
- 2) Our third goal was impacted by difficulty determining the method to implement the QR codes, which is now decided and considered to be implemented. We will be looking into the integration of the QR codes next week, and how OpenCV will streamline the project.

What are the goals for next week?

< Enumerate your goals for next two weeks. Be sure that your goals are measurable and connected to your requirements. >

- 1) Successfully test the environment and begin ADL task code testing to 3 key poses (floor, table, light switch). Implement skeleton behaviors for the three tasks and successfully run each.
- 2) Get a functional base layout for QR code implementation using OpenCV and AprilTags that can detect at least 3 tags that return their poses into ROS2.

- 3) Design at least 2 basic 3-D printed models for objects to be used during physical testing.
- 4) Investigate UI strategies for app-based voice command integration of the ROS2 Python code. Provide two options for mapping.

How many hours were spent on each goal noted above in the past two weeks?

< For each person on the team list the goals they worked on including completed and non-completed goals>

Team Member	Goal	Completed?	Hours Spent
Allison	1	YES	10 mins
Oliver	1	YES	20 mins
Brielle	1	YES	10 mins
Jadon	1	YES	20 mins
Allison	2	YES	0 hours
Oliver	2	YES	5 hours
Brielle	2	YES	0 hours
Jadon	2	YES	3 hours
Allison	3	NO	0 hours
Oliver	3	NO	30 mins
Brielle	3	NO	0 hours
Jadon	3	NO	0 hours

Status Report for CARRT Group for Week #5

What were the goals for this two-week period?

- 1) Successfully test the environment and begin ADL task code testing to 3 key poses (floor, table, light switch). Implement skeleton behaviors for the three tasks and successfully run each.
- 2) Get a functional base layout for QR code implementation using OpenCV and AprilTags that can detect at least 3 tags that return their poses into ROS2.
- 3) Design at least 2 basic 3-D printed models for objects to be used during physical testing.
- 4) Investigate UI strategies for app-based voice command integration of the ROS2 Python code. Provide two options for mapping.

What goals were accomplished in this two-week period?

- 1) This goal was partially accomplished. We successfully tested the simulation environment and verified robotic arm movement through Python and ROS2 nodes. Skeleton behaviors were implemented, but full execution of all three ADL pose sequences was not completed.
- 2) This goal was completed. We developed a functional QR detection layout using OpenCV and AprilTags. Multiple tags were printed and successfully detected.
- 3) Our third goal was completed, we designed a basic 3-D printed model of a table and a bottle that will be used during our physical ADL testing.
- 4) Our fourth goal was completed. After evaluating different UI strategies for the app-based voice command integration, we selected a ROS2-native UI node.

Reflect critically on any goals not accomplished.

We did not complete the full ADL execution across all three key poses due to time spent stabilizing the arm movement and troubleshooting. Now that the environment is functioning reliably, we will shift focus toward refining task scripts and improving execution consistency.

What are the goals for next two weeks?

- 1) Achieve successful execution of at least 2 of the 3 ADL tasks in simulation with a 30-50% success rate.
- 2) Prepare for the design review presentation with PowerPoint and scripts ready.
- 3) Design tablet UI wireframes

How many hours were spent on each goal noted above in the past two weeks?

Team Member	Goal	Completed?	Hours Spent
Oliver	1	YES	4 hours
Allison	1	YES	-
Jadon	1	YES	-
Brielle	1	YES	-
Oliver	2	YES	-
Allison	2	YES	3 hours
Jadon	2	YES	-
Brielle	2	YES	-
Oliver	3	YES	30 minutes
Allison	3	YES	-
Jadon	3	YES	-
Brielle	3	YES	-
Oliver	4	YES	-
Allison	4	YES	-
Jadon	4	YES	-
Brielle	4	YES	1-2 hours

Status Report for CARRT Group for Week #7

What were the goals for this two-week period?

- 1) Achieve successful execution of at least 2 of the 3 ADL tasks in simulation with a 30-50% success rate.
- 2) Design tablet UI wireframes

What goals were accomplished in this two-week period?

- 1) Our second goal has had major progress, where group member Brielle has worked on setting the blueprints for the implementation. We worked to get in contact with fellow students with extra background on UI, and have the wireframe for the UI successfully distributed to group members.

Reflect critically on any goals not accomplished.

< Explain why you did not complete any missed goals and what you plan to do moving forward with respect to these goals. >

- 1) Our first goal was too far-ahead in thinking, and we should have considered the many steps to proper testing. We have created a working directory of functions for the arm, and are working on creating a functional model utilizing those different parts. However, considering we are far from our next video demo, there is plenty of time to complete this first goal, perhaps with a higher success rate, and we will begin looking into real-world testing on the side during continued simulation.

What are the goals for the next two weeks?

< Enumerate your goals for next two weeks. Be sure that your goals are measurable and connected to your requirements. >

- 1) Successfully test at least two of the three selected ADL tasks using the Genova Kin 3 ARM in the Gazebo simulator.
- 2) Begin developing the user interface for the Genova Kin 3 ARM and determine a method to project it onto an iPad for interaction.

How many hours were spent on each goal noted above in the past two weeks?

< For each person on the team list the goals they worked on including completed and non-completed goals>

Team Member	Goal	Completed?	Hours Spent
Allison	1	NO	-
Oliver	1	NO	5 hours
Brielle	1	NO	-
Jadon	1	NO	6 hours
Allison	2	YES	1 hour
Oliver	2	YES	-
Brielle	2	YES	3 hours
Jadon	2	YES	-

CARRT Assistive Arm CSE Project — Weekly Status Report #8

Current Report Period: Weeks #6-8

1. Goals for the Current Period?

Carry-Over Goals (unaccomplished from prior reporting periods)

1. **Gazebo ADL Simulation Stabilization, Week #5** – [☒ | ☐] – Stabilize current code and achieve successful, repeatable simulation with a 30-50% success rate over multiple tests.
 - a. **Reason for Carry-Over:** The ADL execution was not completed by Week 5, due to time being spent rebuilding the motion planning package after Python packages were notably missing from ROS2 Jazzy software. This left little time remaining to do task scripting and testing.
 - b. **Approach:** With the environment and planning library now stable, the ADL task scripting may be further distributed over team members. The building of each ADL task will be completed using helper_moves, vision_stub, and scene testing infrastructure.

New Goals

2. **Design Review Presentation** – [☒ | ☐] – Prepare and deliver the Design Review presentation in the class, with a PowerPoint and rehearsed scripts for Week 6.
 - a. **Approach:** Divide slides by project component, and rehearse prior to delivery of presentation. Completed as expected.
2. **Tablet UI Wireframes** – [☒ | ☐] – Design a wireframe based off of previous design choices, to implement the ROS2 packages. Capture layout, interaction flow, and voice command triggers (button press).
 - a. **Approach:** Wireframe creation led by Brielle, based on previously decided strategy from last period. Shared drafts with team before finalization.
3. **Specification Document** – [☒ | ☐] – Complete and submit the Specification document as required, covering system design, constraints, trade-offs, risk analysis, and traceability to requirements and needs.
 - a. **Approach:** Divide sections by component ownership. Oliver leads architecture and design (Brielle for UI node), full team reviews and contributes before submission.

2. Goals Accomplished in the Period?

1. **Task #1** – [Accomplished / **Partially**]: The core motion planning library was fully rebuilt using the MoveGroup action server interface, rather than the moveit_py file due to dependency issues in the environment. Coding components: apriltag_key, static_scene, vision_stub, and scene_from_vision were completed and verified in simulation. The clear_table task node is actively in development, and has successfully grasped only some of a set of objects, and so full repeatable execution of the tasks has yet to be achieved.
 - a. **Completed By:** Oliver (~8hrs), Jadon (~6hrs), Allison (~2hrs)
2. **Task #2** – [**Accomplished** / Partially]: The Design Review PowerPoint was completed and the presentation delivered in class during Week 6.
 - a. **Completed By:** Entire Team (~2hrs each)
3. **Task #3** – [**Accomplished** / Partially]: The tablet UI wireframes were completed and distributed to the group members through GitHub. Brielle led wireframe design and development, collaborating with additional students with UI background to finalize layout and interaction flow. Also worked on button/control-mapping using voice and button commands.
 - a. **Completed By:** Brielle (~3 hrs)
4. **Task #4** – [**Accomplished** / Partially]: The specification document was completed and submitted on March 6, 2026. The document was written to cover all specifics of our design, from entire design implementation to trade-offs and otherwise.
 - a. **Completed By:** Oliver (~3 hrs), Allison (~1hr), Jadon (~1hr), Brielle (~2 hrs)

3. Critical Reflection on Unaccomplished Goals for the Period

1. **Task #1** – Full execution was not achieved: The team underestimated the number of intermediate engineering steps that would be necessary to complete task execution. The MoveIt Python bindings library was not available on Jazzy, and we were forced to change to the MoveGroup action server method, rewriting a major block of code. We also needed scene planning infrastructure in order to test in the simulator.
 - a. **Revised Approach:** With infrastructure now working as intended, ADL scripting will be the sole focus over the next period. Tasks will be divided over group members. We will again target a 30-50% execution success rate before the next demo.

4. Major Goal List for Next Period (End of Period)

1. **ADL Task Execution** – Continue implementation and successfully test 2 of 3 ADL task scripts using Rviz and MoveIT. Target a repeatable execution success rate of 30-50%.
 - a. **Approach:** Explain plan of action / approach.
2. **UI Node Integration** – Begin integration of the UI node with the ROS2 tasks, verifying the task selection from the UI correctly triggers the ADL task execution.
 - a. **Approach:** Brielle connects the UI wireframe to the /adl_command topic, and test with a manual publish to confirm the task node receives and responds to commands.
3. **Test Plan Document** – Design and submit a test plan document to define key metrics and procedures on project testing.
 - a. **Approach:** Each team member works to draft the criteria for the ADLs and consolidate into one file.

5. Work Division for the Team (Over Entire Period)

- Divisions are specifically noted in Sections 2 and 3. For each team member, by total hour count for the current week:

Name:	Oliver Gaston	Jadon Orr	Allison Moline	Brielle Ashmeade
Hours Total:	~14 hrs	~9hrs	~5hrs	~7hrs

CARRT Assistive Arm CSE Project — Weekly Status Report #11

Current Report Period: Weeks #10-12

1. Goals for the Current Period?

Carry-Over Goals (unaccomplished from prior reporting periods)

1. **Physical Arm Testing, Week #11** – [☒ | ☐] – Successfully run 1/3 of the ADL tasks using the arm's physical hardware.
Reason for Carry-Over: Execution on the physical robotic arm hardware was not completed because of a last-minute transition from the WSL environment to an Ubuntu virtual machine, which led to unexpected issues when attempting to control the arm.
 - a. **Approach:** Now that the team has a better understanding of the Ubuntu virtual machine environment, we will proceed with running and testing the physical hardware through the Ubuntu terminal.

New Goals

2. **Rough Draft of Paper to Publish** – [☒ | ☐] – Prepare and deliver a rough draft of the academic paper describing our project, methodology, and results.
 - a. **Approach:** Compile and analyze the results obtained from our experiments, along with previously developed project documentation. These materials will be organized into a structured draft that includes an introduction, methodology, results, and discussion sections to clearly communicate our work.
3. **Update Specification Document** – [☒ | ☐] – Update specification document based on corrections gave to us by Dr. Christensen
 - a. **Approach:** Review the corrections Dr. Christensen provided and make necessary edits to ensure that our document is accurate.

2. Goals Accomplished in the Period?

1. **Task #1** – [**Accomplished** / Partially]: Successfully finish code for 3 ADL tasks and have a 80% success rate during simulation for each task. Complete testing logs for the testing report for each ADL task.
 - a. **Completed By: Oliver (~88hrs), Jadon (~20 hrs)**
2. **Task #2** – [**Accomplished** / Partially]: Completed a project demonstration with Dr. Christensen and team, showcasing system functionality and results.
 - a. **Completed By: Entire Team (~2hrs each)**
3. **Task #3** – [**Accomplished** / Partially]: The tablet UI wireframes were completed and distributed to the group members through GitHub. Brielle led wireframe design and development, collaborating with additional students with UI background to finalize layout and interaction flow. Also worked on button/control-mapping using voice and button commands.
 - a. **Completed By: Brielle (~3 hrs)**

3. Critical Reflection on Unaccomplished Goals for the Period

1. **Task #1** – Full execution of physical arm was not achieved: This issue occurred due to an unexpected transition from WSL to a virtual machine running Ubuntu, which disrupted communication between ROS2 nodes and the robotic arm.
 - a. **Revised Approach:** With an improved understanding of ROS2 communication within the Ubuntu virtual machine environment, we will pivot testing the physical arm with enhanced debugging.

4. Major Goal List for the Week

1. **ADL Task Execution** – Continue to test 3 of 3 ADL tasks using physical hardware with a success rate of 80%.
 - a. **Approach:** Continue debugging to ensure reliable communication between ROS2 nodes and the robotic arm. Then begin testing physical hardware.
2. **Speed Up Arm Execution** – Increase the execution speed of the robotic arm during ADL tasks to ensure performance comparable to a human-operated arm.
 - a. **Approach:** Do additional testing and debugging while increasing arm speed to ensure motion planning is successful, and simulation still has a success rate of 80%.

5. Work Division for the Team (Over Entire Period)

- Divisions are specifically noted in Sections 2 and 3. For each team member, by total hour count for the current week:

Name:	Oliver Gaston	Jadon Orr	Allison Moline	Brielle Ashmeade
Hours Total:	~95 hrs	~20 hrs	~5hrs	~7hrs

CARRT Assistive Arm CSE Project — Weekly Status Report #13

Current Report Period: Weeks #11-13

1. Goals for the Current Period?

Carry-Over Goals (unaccomplished from prior reporting periods)

1. **Physical Arm Testing, Week #12** – [☒ | ☒] – Successfully execute all 3 ADL tasks using the arm's physical hardware.
Reason for Carry-Over: While the system is now connected to the Ubuntu machine basic movement (home position) is functional, the ADL tasks still do not execute and stop before the first movement due to issues in object detection and system integration.
 - a. **Approach:** Our team will continue testing and debugging the integration between vision, scene generation, and motion execution to enable full ADL task execution.
2. **Paper Development, Week #12** – [☒ | ☐] – Continue refining and formatting the academic paper
Reason for Carry-Over: The paper draft is still in progress, still requires additional formatting and refinement.
 - b. **Approach:** Our team will continue editing and improving the document for submission.

New Goals

- Complete Paper for Submission** – [☒ | ☐] – Finalize the academic paper for submission
 - a. **Approach:** Complete final edits, formatting and ensure the paper meets IEEE guidelines
- Begin Poster Development** – [☒ | ☐] – Start designing the project poster for final presentation.
 - b. **Approach:** Outline poster sections and begin drafting visuals and key content based on the paper.
- ADL task Debugging** – [☒ | ☐] – Identify and fix issues preventing full ADL task execution.
 - a. **Approach:** Focus on improving object placement and execution flow so tasks can complete.
- System Testing on Physical Hardware** – [☒ | ☐] – Improve consistency and reliability of system performance
 - a. **Approach:** Perform repeated testing to validate stable behavior and accuracy.

2. Goals Accomplished in the Period?

1. **Task #1** – [**Accomplished** / Partially]: Worked on draft of paper to publish
 - a. **Completed By:** Oliver (~7hrs), Jadon (~2hrs), Allison(~2hrs), Brielle(~2hrs)
2. **Task #2** – [**Accomplished** / Partially]: Testing a debugging of physical robotic arm movement
 - a. **Completed By:** Oliver (~13hrs), Jadon (~5 hrs)

3. Critical Reflection on Unaccomplished Goals for the Period

1. **Task #1** – Execution of physical arm was partially successful; however, the ADL tasks could not fully run: The system now demonstrates smooth movement and improved pose accuracy, but tasks fail before completing due to remaining issues in object placement and system integration.
 - a. **Revised Approach:** Our team will continue debugging object placement and ensuring consistency between scene geometry and grasp targets through testing and calibration movement

4. Major Goal List for the Week

1. **ADL Task Execution** – Continue to test 3 of 3 ADL tasks using physical hardware with a success rate of 80%.
 - a. **Approach:** Focus on improving object placement and execution reliability
 2. **Finalize Paper** – Complete and prepare the academic paper
 - a. **Approach:** Finish formatting and apply final edits
 3. **Poster Development** – Begin designing the project poster
 - a. **Approach:** We will create the layout and draft the key content
- Draft other Submission Documents** – Plan and draft the remaining submission documents
- a. **Approach:** We will continue to use the academic resources provided by Dr. Christensen and create rough drafts for the remaining submission items for the final submit.

5. Work Division for the Team (Over Entire Period)

- Divisions are specifically noted in Sections 2 and 3. For each team member, by total hour count for the current week:

Name:	Oliver Gaston	Jadon Orr	Allison Moline	Brielle Ashmeade
Hours Total:	~ 20 hrs	~ 7 hrs	~2hrs	~2hrs

Summary of New Knowledge Learned

The CARRT Assistive Robotic Arm project introduced a partially functional prototype of a wheelchair-mounted robotic assistant for ADL tasks. We worked with a robotic arm, wrist-mounted camera, ROS 2, MoveIt 2, AprilTag detection, and a tablet-accessible interface for three ADL tasks: clear table, pick up dropped bottle, and medication handoff after verification.

The project successfully developed the core architecture for this project in ROS 2, with separate layers for user interaction, perception, planning-scene management, motion planning, task logic, and safety/recovery actions. We also developed substantial documentation and testing plans to support the design, such as the various course required documents: specification, test plan, and other materials.

However, our implementation was not able to fully achieve the reliable end-to-end execution of all ADL tasks. The physical arm could be connected and commanded, and the system demonstrated basic motion and accuracy, but the complete workflows remained inconsistent throughout development. The main unresolved issues were caused from integration difficulties in perception, object pose estimation, and planning-scene editing, and object targeting. These problems appeared small, but significantly effected the ability to successfully grasp the objects successfully, due to the AprilTag-to-base_link transformation having slight errors.

Our final result is best described as a research prototype, strengthening our understanding of design and physical validation and testing. The system, while only partially implemented, demonstrates the feasibility of the selected architecture, but would need more work to be a reliable assistive system. Future development would require further testing, calibration, and improved camera and cable management, as well as stronger failure recovery, and stronger systematic validation of each ADL.

The most important knowledge gained from this project was practical testing experience on the robotic system's integration, ROS 2 communication, MoveIt motion planning, AprilTag perception, and accessibility-focused UI design. We also greatly learned about the importance of safety constraints, and the importance of following timelines to avoid delays that cause failure, and working as a team.

Design and Failure Analysis of a ROS 2-Based Assistive Robotic Arm Prototype for Daily Living Support

Oliver Gaston
Bellini College of AI, Cybersecurity,
and Computing
University of South Florida
Tampa, Florida
oliver.rain.gaston@gmail.com

Allison Moline
Bellini College of AI, Cybersecurity,
and Computing
University of South Florida
Tampa, Florida
allisonm1317@gmail.com

Jadon Orr
Bellini College of AI, Cybersecurity,
and Computing
University of South Florida
Tampa, Florida
jadonorr78@gmail.com

Brielle Ashmeade
Bellini College of AI, Cybersecurity,
and Computing
University of South Florida
Tampa, Florida
brielle.ashmeade1@gmail.com

Abstract—Individuals with reduced upper-limb mobility may require assistance to perform Activities of Daily Living (ADLs), including tabletop organization, dropped-object retrieval, and medication handling. This paper presents a wheelchair-mounted assistive robotic manipulation prototype intended to allow a user to initiate ADL tasks through an accessibility-oriented tablet interface equipped with button and voice-command support. The prototype integrates a Kinova Gen3 7-DoF robotic arm, Robotiq 2F-85 gripper, ROS 2 middleware, MoveIt 2 motion planning, and AprilTag-based perception to detect, localize, and manipulate tagged household objects within a structured workspace. Three ADL workflows were implemented: clearing table objects, retrieving a dropped water bottle, and medication hand-off with user-name verification through the UI. Across the last seven physical clear-table test days, log analysis counted 70 strict full-task successes from 538 clear-table command starts, for a 13.0% strict command-start success rate, with object-level clearing of 155/298 objects, or 52.0%. Additional late medication-handoff testing on April 29-30 counted 13 successful hand-offs from 80 command starts, or 16.3%, with the final day improving to 3/13, or 23.1%. These results show partial manipulation capability but not reliable end-to-end ADL completion. The main observed limitations were perception-to-motion integration, object-pose alignment, planning-scene accuracy, grasp-position offsets, constrained workspace geometry, wrist-camera reliability, and incomplete terminal-state logging. The paper therefore presents the system as an early-stage assistive robotics prototype and analyzes the integration failures encountered while transitioning from modular implementation to physical ADL execution.

Keywords—*assistive robotics, activities of daily living (ADLs); wheelchair-mounted manipulator; AprilTag-based perception; accessible human-robot interface; ROS 2; motion planning*

I. INTRODUCTION

Activities of Daily Living (ADLs) such as retrieving dropped objects, organizing a workspace, and handling medication handoff can become difficult or impossible for individuals with limited upper-limb mobility to complete independently. Prior assistive robotics literature frames these systems as human-centered platforms requiring attention to user interaction, autonomy level, manipulation reliability,

and deployment constraints, rather than arm motion alone [1]. For this project, those concerns connect directly to the accessibility interface in Section III-C, perception and planning pipeline in Sections III-D and III-E, and failure analysis in Section V.

In this work, we developed a wheelchair-mounted assistive manipulation prototype using a Kinova Gen3 arm for three representative ADLs: clear table, pick dropped bottle, and medication handoff with user-name verification. The system combines a tablet interface with button and voice-based control, AprilTag-based perception for object localization, ROS 2 communication, and MoveIt 2 planning and execution [6], [9], [10], [12]–[16]. The implemented task workflows are summarized later in Table 2, while the evaluation procedure and failure categories are defined in Section IV.

Compared with prior wheelchair-mounted assistive platforms and shared-autonomy approaches, this work focuses on the integration challenges involved in combining accessibility-oriented interaction, AprilTag-based perception, MoveIt 2 planning, and task-level ADL logic on a physical Kinova platform [2]–[5]. Specifically, this paper contributes: (1) a modular ROS 2 software architecture that separates interface, perception, planning-scene management, motion execution, and ADL task logic; (2) the implementation of three structured ADL workflows on a wheelchair-mounted Kinova platform; and (3) a late-stage physical-log analysis that distinguishes full ADL completion from intermediate subsystem and step-level progress. This distinction is central to the paper because the prototype was not a completed assistive device; rather, it was an integration-limited system in which several subsystems operated but did not remain reliable enough across the full perception-to-action sequence.

The remainder of the paper is organized as follows. Section II reviews related assistive manipulation systems and positions the project relative to prior wheelchair-mounted and vision-based ADL work. Section III presents the hardware, software, perception, planning, UI, and ADL-task implementation. Section IV defines the intended experimental setup and evaluation metrics. Section V reports observed prototype behavior and failure modes. Section VI discusses the implications of those failures, and Section VII concludes with future work.

II. RELATED WORK

Recent work has considered physically assistive robotics as a broad research issue and identified recurring design themes including user interaction methods, control modality, levels of autonomy, and adaptation to user needs. Nanavati et al. provide a systematic review that frames physically assistive robots as human-centered systems involving accessibility, workload reduction, and practical deployment constraints, not simply manipulation and control [1]. This framing motivates the project architecture in Section III, where the UI, perception, planning, and task logic are deliberately separated to support structured integration.

Other closely related systems include wheelchair-mounted robotic arms for ADL support. Sunny et al. present a 6-DoF wheelchair-mounted assistive manipulator controlled through eye-gaze interaction, demonstrating the importance of alternative control modalities for users with limited motor function [2]. Chi et al. evaluate a wheelchair-mounted robotic arm for household tasks and emphasize practical task execution and user-facing assistive design [3]. These studies align with our goal of wheelchair-mounted ADL assistance, while our prototype differs by using a tablet-and-voice interface and a marker-based perception pipeline rather than gaze control or demonstration recording.

Other prior work provides useful contrasts in autonomy and perception strategy. Jain et al. use shared autonomy through a body-machine interface, illustrating a different balance between user command and system-level autonomy than the direct command structure used in this project [4]. Shahria et al. focus on vision-based manipulation for ADL assistance, offering a useful comparison for the perception-to-manipulation pipeline described in Sections III-D and III-E [5]. In contrast, our system uses AprilTag-based localization in a constrained workspace to prioritize object identity, pose repeatability, and integration simplicity [12]-[16]. The limits of that design choice are revisited in Section VI-B.

III. SYSTEM DESIGN / METHODS

A. Hardware Platform

The system is implemented as a wheelchair-mounted assistive manipulator for structured tabletop and floor manipulation. The current deployment assumes a fixed lab configuration, and this environment is hardcoded into the planning scene as static collision geometry, including the wall, desk, table, wheelchair, and placement regions such as the shelf and bin. This fixed-scene assumption is important to the planning-scene discussion in Section III-E and becomes one of the major failure sources in Sections V-E and VI-C. The arm base is mounted on the left side of the wheelchair, and the system expects initialization from a valid predefined Kinova pose, such as Retract or Home.



Figure 1: Wheelchair-mounted Kinova Gen3 robotic arm at predefined Retract pose

The manipulation hardware consists of a Kinova Gen3 7-DoF arm and a Robotiq 2F-85 gripper [6], [7]. Workspace layout and object placement were selected so task-relevant objects would remain within the reachable and collision-aware operation described in Section III-E. Motion commands were executed using MoveIt planning/execution and direct joint-trajectory control, as summarized in Table 1 and described in Section III-E [10].

Perception is provided by an Intel RealSense D435 camera mounted at the robot's wrist [8]. The AprilTag pipeline subscribes to the camera image stream, estimates a 6-DoF tag pose, and transforms detections into the robot base frame for MoveIt manipulation planning [12]-[16], [20]. Tag visibility is assumed to be sufficient for operation, with line of sight to the robot and suitable lighting. Camera calibration parameters are fixed across trials, which later becomes a key limitation in Sections V-E and VI-B.

The static environment is represented in the robot-centric coordinates by using measurements referenced from the Kinova arm's base frame for consistency. The task table (IKEA LACK, 55cm x 55cm x 45cm) is positioned with its front edge approximately 16 inches from the base_link center, centered with respect to the manipulator. A nearby desk obstacle is modeled on the left side at approximately 20 inches. For floor-retrieval operations, the mounted base height is approximately 14 inches off the ground. The manipulated object set contains 5 household items with simplified geometry classes (cylindrical or box-like) and object dimensions are connected to AprilTag IDs for planning scene and grasp logic.

Runtime execution uses a distributed architecture to combine robot control and user interface components. The robot stack, including ROS 2 nodes for planning, task logic, and perception, runs on Linux with ROS 2 Jazzy [9]. The UI runs as a tablet-accessible web interface and displays command and status messages through ROS-compatible bridge nodes. Communication between components uses local network transport, with camera, planning, and task

nodes coordinated through ROS topics and services. Since the system is interaction-driven rather than continuously operating, command-response delay and camera processing latency were treated as important design factors, consistent with the ROS 2 communication and QoS concerns described in [18].

Safety was enforced through both geometric constraints in the planning scene and runtime policy. Conservative motion limits were applied near user-adjacent regions, and emergency-stop behavior was treated as a system-level command. These choices were informed by robot safety and functional-safety standards, including ISO 10218-1, ISO/TS 15066, and IEC 62061 [22]-[24]. Collision avoidance depends on planning-scene fidelity and emergency-stop reliability, which connects this design section to the observed planning-scene and safety limitations discussed in Sections V-E, VI-C, and VI-E. For medication handoff, interaction is constrained to a restricted delivery zone with controlled approach and retreat behavior.

Component	Implementation
Robotic Arm	Kinova Gen3, 7-DoF wheelchair-mounted manipulator
Gripper	Robotiq 2F-85 parallel gripper
Camera	Intel RealSense D435 Depth Camera
Middleware	ROS 2 Jazzy on Ubuntu 24.04
Motion Planning	MoveIt 2 + direct joint control
Perception	AprilTag-based pose estimation with TF2 to base_link
User Interface	Flask-based web UI (button/voice)
Core Task Nodes	<code>'clear_table'</code> , <code>'pick_dropped_bottle'</code> , <code>'give_medication'</code> using <code>'adl_controller'</code>
Vision / Scene Nodes	<code>'vision_apriltag'</code> / <code>'vision_stub'</code> , <code>'scene_from_vision'</code> , <code>'scene_static'</code> , <code>'joint_state_sanitizer'</code>

Table 1: System Implementation Summary of components and used software

B. Software Architecture

Our software stack is implemented as a set of modular Python files for a ROS 2 Jazzy application, organized into two project packages: `adl_tasks`, holding the runtime nodes and necessary helpers, and `adl_interfaces`, which provides custom service interfaces for perception queries and task coordination [9], [11]. In the current build, system startup is managed using `adl_start.launch.py` bringup, which combines the per-node execution into a single wrapper. This launch path starts the static-scene publisher, joint-state sanitizer, perception nodes, vision-to-scene bridge, task nodes, system controller, and tablet-accessible UI.

The architecture is divided into functional layers, as shown in Figure 4 in Appendix A. At the top layer, the Flask-based `adl_ui` provides task selection, publishes user commands, and displays status information. The control layer contains the three ADL tasks and `adl_controller`, which provides shared behavior such as IDLE parking, turn-off handling, and emergency-stop across ADL tasks. The perception and scene-management layer processes wrist-camera detections through either `vision_apriltag` or `vision_stub`, publishes visible tag IDs, serves pose requests

through `get_tag_pose`, and converts valid detections into MoveIt planning-scene collision objects through `scene_from_vision`. This layered design is later evaluated through the integration failures summarized in Section V-E.

Below the control and perception layers is the motion-planning layer, implemented through `helper_moves.py` and shared by all task nodes. General task motions use MoveIt 2 planning and action-based trajectory execution, while repeatable fixed-posture movements such as table scan, floor scan, home, and retract may use direct joint-trajectory execution to improve viewpoint repeatability and reduce inverse-kinematics variation [10]. This hybrid planning strategy is connected to the task stages in Section III-F and the planned test metrics in Section IV-C. The failure analysis in Section V shows that repeatable scan postures helped integration, but did not eliminate calibration and object-offset failures.

C. User Interface

The system's user interface was implemented as a tablet-accessible web application that provides a simple front end for commanding the assistive arm during ADL execution. The UI was designed around accessibility-oriented goals, including large task-selection buttons, status feedback, and voice-command support. These interface goals are aligned with general web accessibility principles such as operability, understandability, and robustness in WCAG 2.2 [21]. The interface connects directly to `/adl_command` and `/task_status`, which ties this section to the ADL-task definitions in Section III-F and evaluation criteria in Section IV-B.

The interface sends signals to the robot stack through a shared ROS 2 command and status branch. Task requests are first sent through the common command topic, allowing the UI to trigger the tasks without requiring direct connections to each task's internal logic. Additionally, the UI also has various system controls, such as "Turn Off" and "Emergency Stop" which both prepare the device for a system shutdown, either in emergent or normal settings. These included system actions are coordinated through the controller layer to allow safety-critical commands to be handled consistently across all ADLs or system statuses, directly avoiding disconnected handling to fork intended behavior.



Figure 2: ADL Support System UI

To support safety and usability, the UI also presents a live task-status to allow the user to be aware of the arm's active goals. This display details both the current execution state of the task as well as associated details with the status, to allow the user to distinguish if the arm is currently either: idle, running, succeeded, failed, or cancelled in status. This

feedback allows the user to understand multi-stage actions across tasks, such as scanning, detection, grasping, transportation, and aligning to placement or handoff. Explicit state feedback also helps the user to understand whether the system requires input, is actively executing, or has terminated due to a failure or cancellation.

In medication hand-off, the UI is uniquely used to collect the user's name and compare it against the medication bottle's known patient. If the user does not identify themselves as the patient, the task self-terminates, returns to Retract, and cancels the grasp-and-place action. This confirmation step provides a functional and safety purpose by keeping the user involved in the decision while preventing an autonomous handoff under a name mismatch. The corresponding workflow is summarized in Table 2 and later discussed in Sections V-D and VI-E.

Overall, the user interface is the main accessibility layer between the user and the assistive robotic manipulation. Its design emphasizes low-effort task initiation, clear execution feedback, explicit confirmation for medication delivery, and immediate access to safety controls. Together, these allow us to target our larger goal of enabling wheelchair-mounted assistive manipulation through a simple and understandable user interface that is accessible for users with reduced upper-limb mobility.

D. AprilTag-Based Perception with RealSense Camera

The perception system uses AprilTag fiducial markers to enable manipulable object detection and localization for the ADL tasks. AprilTags from the tag36h11 family consist of a binary pattern with a border that encodes a unique tag ID and supports pose estimation [12]-[15]. These tags are attached to relevant household objects so that each target can be uniquely identified by the vision system. A wrist-mounted Intel RealSense D435 camera provides the input image stream used for tag detection [8]. According to the project design, the active camera feed is received on `/wrist_mounted_camera/image`, and the perception pipeline was designed to support real-time object updates for task execution, connecting this section to the metrics in Section IV-C.

For each incoming image, the system detects visible AprilTags and estimates a 6-DoF pose for each tag. These poses are transformed from the camera frame into the robot's `base_link` coordinate frame using TF2 and REP-105 frame conventions, allowing manipulation tasks to plan motions in the robot's reference frame [20]. The vision node then publishes visible tag IDs on `/detected_tag_ids` and provides a `GetTagPose` service, allowing task nodes to request the most recent pose of a specific tag. This interface keeps task nodes independent of whether they use simulation stub vision or the real AprilTag vision node. The same transformation chain later becomes one of the main observed failure points in Sections V-E and VI-B.

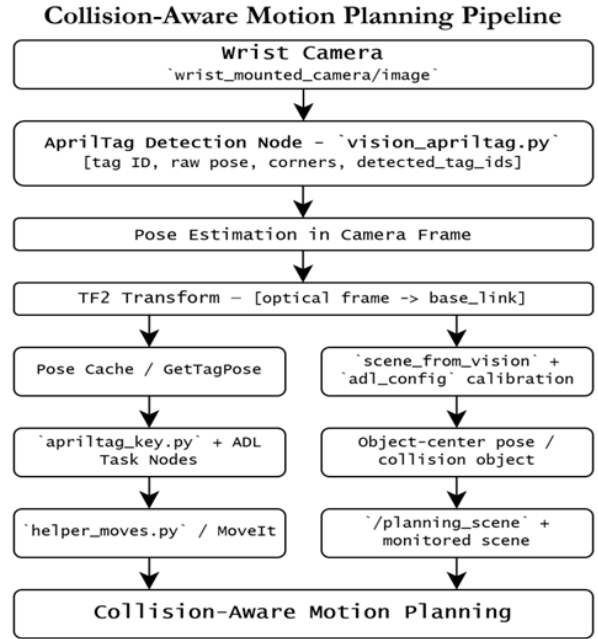


Figure 3: Decision pipeline for camera image to AprilTag-based pose localization and motion planning

E. Motion Planning and Manipulation

Our motion-planning and manipulation layer is implemented using a shared motion-helper layer used in all three ADL task nodes. The helper layer provides a common base interface for pose goals, joint goals, Cartesian approaches, gripper commands, fixed postures, and recovery behavior. This prevents each ADL task from implementing its own motion logic and keeps grasp, place, and recovery behavior consistent across the system. This section provides the motion basis for the ADL workflows in Section III-F and the failure categories in Section V-E.

Generally, arm motion is planned and executed using MoveIt 2 action-based planning requests [10]. Pose-based motions are handled by the task nodes, which create a target end-effector pose in the `base_link` frame and submit it through the shared helper. The helper creates a motion-planning request with workspace bounds, planning-time limits, velocity and acceleration scaling, and position/orientation tolerances. These values are expressed using standard ROS units and coordinate conventions, consistent with REP-103 and REP-105 [19], [20]. The implementation also checks for fresh and valid joint-state information prior to planning to reduce errors caused by stale or incomplete joint data.

The planning scene contains both static and dynamic collision-object geometry. Static objects are published at startup and represent the known fixed lab environment, such as the wheelchair, table, floor, and surrounding walls or obstacles. Dynamic objects are the specified AprilTag objects detected by the vision-to-scene pipeline. After a tag pose is transformed into the `base_link` frame, `scene_from_vision.py` converts the tag pose into an object-center pose with object-specific geometry and calibration offsets, then publishes the associated object into the MoveIt planning scene [10]. This design connects directly to the planned scene setup in Section IV-A and the planning-scene mismatch discussed in Section VI-C.

Additionally, aside from MoveIt planning, the system also uses deterministic fixed joint postures for repeatable scanning and parking behavior. These poses, such as table scan, side-tag scan, ground scan, home, and retract are all represented as named joint configurations. When utilized, these motions run directly through the joint-trajectory controller, and have pose-verification steps to check if the arm actually completed the move exactly. If direct joint execution fails, the helper falls back to MoveIt. This hybrid approach allowed for better repeatability for the wrist-camera scan viewpoints, and allowed the preservation of general MoveIt-based planning for object pick and place tasks.

Manipulation actions use object-specific grasp parameters. Before grasping, the arm moves to an above pre-grasp pose generated from the object's detected pose. The gripper then opens before the final approach and closes around the object using a configured width, speed, and force based on the known object type and gripper capabilities [7]. After a successful grasp, the task logic lifts the object, transports it to the appropriate destination, and releases it at a placement or handoff pose. Top-grasp objects, such as flat tabletop objects, and side-grasp objects, such as upright containers, use different approach methods and tolerances to account for object shape and tag placement. These object-specific assumptions are later reflected in the failure analysis in Section V-E.

Recovery behavior is handled by both the ADL task controller and the motion helper. On a motion request failure or cancellation, the helper will then begin to cancel active MoveIt or trajectory-controller goals and wait for the arm to settle at retract before allowing additional commands. At the task level, failed or cancelled ADL actions cause a status update and then the controller will attempt to safely park the arm at Retract, with Home as an active fallback if retract is deemed unsafe by the controller. Emergency-stop commands use the same pipeline for motion-halting, but are system-level and can therefore interrupt active execution and transition the robot towards an idle state at any time.

F. ADL Task Definition

The active system implements three ADL task workflows: clear household objects from a table, retrieve a fallen water bottle, and medication hand-off. Each task is implemented as an independent ROS 2 node that listens for a specific command on the shared command topic and executes only when its command string is received. While tasks differ in object types and destinations, they share the same overall structure: move to a scan pose, obtain target poses from perception, generate object-specific grasp and placement poses, and execute manipulation through the shared motion layer. The workflow summary in Table 2 links these task stages to the evaluation procedure in Section IV-B and the prototype-status summary in Section V-F.

The clear-table task is triggered by the command `clear_table` and is designed to remove supported household objects from the table surface. In the current implementation, supported clear-table objects include a cup, TV remote, and cube, using AprilTag IDs 2, 3, and 4. The task first clears remembered objects from the scene state, moves to a fresh table-scan pose, enables vision to collect visible target IDs, and sorts detected objects by distance from the arm. For each object, the task requests the most recent tag pose and

computes grasp and approach poses based on top-grasp or side-grasp geometry. These implementation details connect to the object geometry in Table 4 and the clear-table limitations reported in Section V-B.

The dropped-bottle task is triggered by `pick_dropped_bottle` and targets the water bottle object with AprilTag ID 0. This task starts by moving to a floor-scan pose, using a ground-view posture when available. After the bottle pose is found from the vision system, the task locks the scene and attempts a staged pick-and-place sequence. The bottle is approached from above, grasped using object-specific parameters, lifted above the floor and table, transported to the handoff location at the front right corner of the table, released, and followed by retreat to Retract. The low-workspace constraints of this task are later discussed in Sections V-C and VI-C.

The medication hand-off task is triggered by the `give_medication` command and targets the medication bottle with AprilTag ID 1. This task differs from the other ADLs because medication delivery requires explicit user-name verification before grasp. The task scans for the medication bottle, obtains the prescribed name associated with the detected tag, prompts for the user's name through the UI, and compares the strings using normalized matching. If the names do not match, the task cancels before grasp-and-place and returns the arm to Retract. If verification succeeds, the task proceeds with side-grasp pickup and transport to the handoff location. This safety-oriented workflow connects to the medication-specific results in Section V-D and assistive-use discussion in Section VI-E.

All three ADL tasks use regular status updates through the shared task-status interface. During execution, task nodes use the status to report the running state and detailed execution information on progress, to display on the user interface regarding scanning, waiting, manipulating, or recovery behaviors. Each task also monitors cancellation and emergency-stop states using the shared task base. On cancellation during object transportation, the task will attempt to stop the motion, release or detach the object, and transition to a controlled terminal state. This shared task structure allows the reuse of perception, planning-scene, motion, status, and recovery behavior across ADL tasks, while preserving the uniqueness of each individual task.

ADL Task	Command	Target	Stages
Clear Table	<code>clear_table</code>	Cup, TV remote, Cube (2, 3, 4)	Scan Table → Get Obj Poses → Grasp / Place Objs → Repeat Until None
Pick Dropped Bottle	<code>pick_dropped_bottle</code>	Water Bottle (0)	Scan Floor → Get Obj Pose → Grasp / Place Obj → Release at Handoff Location

Medication Hand-off	give_medication	Medication Bottle (1)	Scan Table → Request User-Name → Move to Approach → Compare Names → Grasp / Place at Handoff, if match, else retreat
----------------------------	-----------------	-----------------------	--

Table 2: ADL Task Summary – Used command string, target object, and stages of each task.

IV. EXPERIMENTAL SETUP

A. Scene Environment and Objects

The experiments on the system were conducted in a structured lab environment built to represent a simplified wheelchair-mounted assistive manipulation workspace. The robot consisted of a Kinova Gen3 arm mounted on the left side of a wheelchair, with task-relevant objects placed within the arm's reachable workspace. The environment was represented in the robot's base_link coordinate frame, so perception outputs, collision objects, grasp poses, and placement goals could be expressed in a shared reference frame, consistent with the base-frame conventions introduced in Sections III-D and III-E [20].

The main tabletop workspace used an IKEA LACK-style table with dimensions of approximately $0.55 \text{ m} \times 0.55 \text{ m} \times 0.45 \text{ m}$. Our current configuration has the front of the table exactly 16 inches in front of the robotic arm's base_link frame. Since the arm base was mounted about 14 inches off the floor, the table surface was about 4 inches above the base_link frame. This table supported the clear-table and give-medication tasks, as well as hardcoded placement regions used during task execution.

The fixed scene also included object-specific placement regions: a small shelf placed at the back left corner of the tabletop, for the cup and cube placement, and a small bin centered to the back of the table, for TV remote dropoff. A near-user handoff location was selected for the medication bottle and water bottle drops, towards the front right of the table. For the dropped-bottle task, the water bottle is expected to be placed on the floor, AprilTag oriented upwards, within the arm's reachable workspace.

Static collision-object geometry was added to the MoveIt planning scene to approximate the environment and prevent collisions, as described in Section III-E. This included the table, floor, shelf, bin, wheelchair, walls, and nearby scene obstacles such as desks. These were modeled as simple box-shaped collision primitives so the planner could reject motions that would intersect known obstacles while keeping the scene manageable for repeated trials. Dynamic collision objects were added for AprilTag-tracked items using the vision-to-scene pipeline after detection. The mismatch between this modeled scene and the physical workspace is one of the failure modes discussed in Sections V-E and VI-C.

Scene Element	Dimensions	Purpose
---------------	------------	---------

Table	$0.55 \text{ m} \times 0.55 \text{ m} \times 0.45 \text{ m}$	Tabletop workspace for clear-table / give-medication
Shelf Region	$0.2 \text{ m wide} \times 0.14 \text{ m deep} \times 0.05 \text{ m high}$	Destination for cube and cup
Bin Region	$0.12 \text{ m} \times 0.12 \text{ m} \times 0.08 \text{ m}$	Destination for TV remote
Handoff Region	Front-right of tabletop	Destination for water bottle and medication bottle
Floor Region	Modeled floor pane at bottom of workspace	Initial region for pick-dropped-bottle task
Wheelchair	Approximately $0.45 \text{ m} \times 0.70 \text{ m}$	Prevent plans from striking the user seated in the wheelchair
Wall / Desk	Box-shaped collision objects	Approximates fixed lab obstacles surrounding the workspace.

Table 3: Testing Environment obstacles and inclusions, and their intended purpose.

The manipulated object set contains five tagged household objects: a water bottle, medication bottle, cup, TV remote, and cube. Each object was uniquely assigned to an AprilTag ID, and an approximate geometric model for grasp planning and collision-scene updates. Cylindrical objects were represented using diameter and height parameters, while box-shaped objects were represented with length, width, and thickness, or simple cube parameters. The tags were placed to support the intended grasp strategy: bottle and tabletop objects were intended to be grasped from above, with tags on the top face, while the medication bottle and cup require a side facing tag configuration for side grasps.

Object	AprilTag ID	Geometry
Water Bottle	0	Cylinder: diameter = 0.060 m, height = 0.200 m
Medication Bottle	1	Cylinder: diameter = 0.060 m, height = 0.090m
Cup	2	Cylinder, diameter = 0.075 m, height = 0.105 m
TV Remote	3	Box: $0.173 \text{ m} \times 0.043 \text{ m} \times 0.022 \text{ m}$
Cube	4	Box: $0.064 \text{ m} \times 0.064 \text{ m} \times 0.064 \text{ m}$

Table 4: Known AprilTag object dimensions and IDs.

B. Evaluation Procedure

A trial-based evaluation procedure was defined for the structured lab environment. Each ADL task was intended to begin with the robotic arm in a predefined safe posture, such as Retract or Home, and with the required task object or objects placed in the expected workspace region. The task would then be initiated through the tablet interface or an equivalent ROS 2 command. During execution, the system would perform the required scan motion, acquire object-pose information from the AprilTag vision pipeline, generate task-specific grasp and placement targets, and attempt to complete the manipulation sequence using the shared motion-planning and gripper-control layers described in Sections III-D and III-E. The planned development path also included simulation and fake-hardware testing in Gazebo before physical trials [25].

For each task, the object placement and start conditions were designed to be reset before the next trial. In the clear-table task, supported objects would be placed on the table surface within the reachable workspace. In the dropped-bottle task, the water bottle would be placed on the floor within the arm’s reachable workspace. In the medication hand-off task, the medication bottle would be placed on the table, and the system would first perform user-name verification before proceeding with hand-off or cancelling and retreating automatically.

A trial was to be counted as successful only if the task reached its intended final state without collision, unrecovered planning failure, unsafe behavior, or manual intervention. However, because stable end-to-end execution was not reached during physical testing, the procedure was used primarily as a failure-analysis framework rather than as the basis for reporting a statistically meaningful task success rate.

Failures were categorized by the stage in which the trial failed. Detection failures occurred when relevant AprilTags could not be found or localized. Planning failures occurred when the robot was unable to generate or execute a valid path to the requested pose, and grasp failures occurred when the object was not accurately acquired after the gripper closed. Placement and hand-off failures occur if the object was dropped, released outside its target region, or failure to move to the target pose. Additional failures included timeouts, cancellations, emergency stops, and hardware interruptions. After each terminal condition, the system would attempt to return the arm to a safe idle or retract posture.

Task	# of Trials	Start Cond	Success Cond	Failure Categories
Clear Table		Cup, remote, cube placed within the workspace	All objects moved to their destinations	Detection, Grasp, Placement, Planning, Timeout
Pick Dropped Bottle		Bottle on floor in workspace	Bottle detected and picked and placed to handoff	Detection, Grasp, Lift, Placement, Planning,
Give Medication		Medication bottle on table, user name through UI	Correct name match to handoff, else abort	Name Mismatch, Name Timeout, Detection, Grasp, Placement, Planning,
Emergency Stop		Command triggered	Active motion stops and moves to idle retract state	Not Received, Delayed Stop, Recovery Failure

Table 5: Success and failure condition descriptions for the ADL tasks and emergency stop.

C. Metrics

The evaluation plan considered both task-level and stage-level metrics. The primary task-level metric was success rate,

defined as the percentage of command attempts in which the ADL task reached its intended final condition without collision, unrecovered planning failure, unsafe behavior, or manual intervention. Because the final prototype did not satisfy the original reliability targets, Section V reports both quantitative command-attempt outcomes and step-level observations rather than presenting the prototype as a completed system. The logged results should be interpreted as late-stage physical testing data gathered during integration and tuning, not as a fully balanced controlled user study.

For each ADL task, a run was considered successful only when the task-specific goal was completed. In the clear-table task, success required the supported visible tabletop objects to be moved to their assigned destination regions. Because this task involved multiple object attempts, both full-task success and object-level successes were considered during testing. Full-task success determined whether the complete sequence was successful, while object-level success measured the fraction of individual object attempts that were detected, grasped, moved, and released successfully. For the dropped-bottle task, success required the bottle to be detected, grasped, lifted from the floor, and transported and released at the user hand-off destination. For the medication hand-off task, success required correct patient-name verification followed by successful grasp and delivery. Name mismatch trials were also considered successful safety behavior, so long as the task was accurately aborted before grasp and place.

State-level metrics were used to identify the cause of failed or partially failed trials. Detection success was considered when the target AprilTag was detected and returned a usable object pose in the base_link frame. Grasp success was counted when the object was securely acquired after the gripper was closed and could be transported without dropping. Placement or handoff success was counted when the object was released within the intended destination region. Collision occurrence was recorded when the robot, object, table, wheelchair, floor, or other obstacles made unintended contact during a trial. Failed trials were assigned a primary failure category, such as detection failure, planning, grasp, placement failure, verification failure, timeout, cancellation, emergency stop, or hardware interrupt.

Metric	Calculation	Purpose
Task Success Rate	Successful trials / total trials	Primary measure of ADL success or completion
Object-Level Success Rate	Successfully moved objects / attempted objects	For clear-table object attempts
Detection Success Rate	Trials with useable object poses / total trials	Measures AprilTag perception reliability
Grasp Success Rate	Successful grasps / attempted grasps	Measures manipulation reliability after detection
Placement Success Rate	Successful placements / attempted placements	Measures final delivery reliability
Collision Occurrence	# trials with unintended contact	Measures safety-relevant failures

Completion Time	Time from command to terminal status	Measures task efficiency
Verification Correctness	Correct medication action / verification trials	Measures name-confirmation behavior
Primary Failure Mode	Dominant failed stage per trial	Supports failure analysis

Completion time was considered as an optional metric, measured from the task command to terminal task status. In this version of the system, completion time was not reported as a final performance metric because incomplete end-to-end execution made reliability and failure mode more important than speed. Future work should report completion time only after the stage-level failures in Section V-E are sufficiently reduced.

V. RESULTS

A. Physical Integration Results

The final system was evaluated as an integrated prototype in a structured laboratory environment using the Kinova Gen3 robotic arm, Robotiq gripper, wrist-mounted RealSense camera, AprilTag-based perception, MoveIt 2 planning, and the tablet-accessible UI described in Sections III and IV. The system was able to connect to the physical robotic platform, command arm movement, execute scan postures, receive user commands from the UI, and demonstrate improved pose behavior during debugging. These outcomes confirmed that the main hardware and software components could operate together at a basic level.

The system did not achieve reliable end-to-end execution of the three planned ADL tasks on physical hardware. Portions of the task workflows were implemented and tested individually, but the complete task sequences did not consistently complete from user command through perception, planning, grasping, placement, and return-to-idle behavior. The final prototype should therefore be understood as a partially functional integrated system rather than a completed assistive manipulation solution. The remainder of this section separates task-level findings by ADL workflow and distinguishes strict full-task success from step-level or object-level progress.

B. Subsystem and Step-Level Results

The results are separated into task-level outcomes and step-level outcomes. Task-level results report whether a full ADL workflow finished successfully from command start to terminal completion. Step-level results report whether major intermediate parts of the pipeline operated even when the full task did not. This distinction is important because the system was not a total implementation failure; rather, several subsystems operated, but failures accumulated across subsystem boundaries.

Across the seven physical test days used for final clear-table quantitative analysis, April 21, 22, 23, 24, 27, 29, and 30, 2026, 538 clear-table command attempts were logged. Of these, 70 reached strict full-task success, corresponding to a command-start success rate of 13.0%. Because strict task-level success does not capture partial manipulation progress, an additional object-level analysis was used for the clear-table workflow. Across logged clear-table object attempts, 155 of 298 object moves succeeded, corresponding to an object-level success rate of 52.0%.

The earlier five-day clear-table subset from April 21, 22, 23, 24, and 27 was retained as diagnostic step-level evidence. In that subset, 308 command attempts were logged, 152 attempts reached a normal task-completion line, 52 fully succeeded, and 100 ended with partial completion. This indicates that command handling, scanning, and object-processing logic often progressed beyond startup, even though full end-to-end task success remained limited.

Pick Dropped Bottle remained the least reliable physical ADL workflow in the available strict-count logs, with 2 logged successful completions from 38 starts, corresponding to a 5.3% task-level success rate. Give Medication improved substantially in the final two test days, with 13 successful hand-offs from 80 command starts, corresponding to a strict success rate of 16.3%. The final-day medication result was stronger after the user-name verification stage: 3 of the 4 trials that reached UI name input completed hand-off, corresponding to a post-verification success rate of 75.0%. These medication results are reported separately from the five-day clear-table diagnostic subset because the main medication testing occurred during the final two testing days.

Workflow	Task Starts	Full Successes	Task-Level Success Rate	Step-Level Notes
Clear Table	538	70	13.0 %	155/298 object moves succeeded, or 52.0%; earlier five-day subset had 100 partial completions
Pick Dropped Bottle	38	0	0.0%	Some runs reached detection/manipulation stages, but reliability remained low
Give Medication	80	13	16.3 %	Improved from 10/67 on Apr. 29 to 3/13 on Apr. 30; final-day post-verification success was 3/4, or 75.0%

Table 7: Task-level physical results from late-stage command logs.

C. Clear Table Results

The clear-table workflow was the most developed and most thoroughly tested ADL task. It exercised the shared perception, planning-scene, motion-helper, gripper-control, grasping, placement, and recovery infrastructure described in Sections III-D through III-F, and therefore provides the strongest basis for analyzing repeated physical manipulation behavior.

Across the seven-day final testing window, the clear-table task achieved 70 strict full-task successes in 538 command attempts, corresponding to a task-level success rate of 13.0%. A full success was defined as a run that reached normal completion with the selected visible objects cleared. This strict command-start result is lower than the earlier five-

day subset result because the final count includes additional later testing, where more command attempts were logged during continued debugging and tuning.

The earlier five-day subset remains useful for diagnosing where the pipeline progressed. In that subset, 152 of 308 attempts reached a normal task-completion line, meaning that 49.4% of attempts progressed far enough to complete an object-processing cycle. Of those 152 completed attempts, 52 fully succeeded and 100 ended with partial completion. This indicates that the task frequently progressed through command handling, scanning, object selection, and at least part of the manipulation pipeline, even when all selected objects were not cleared.

Date	Full Successes	Command Attempts	Task-Level Success Rate
April 21, 2026	5	74	6.8%
April 22, 2026	14	53	26.4%
April 23, 2026	21	105	20.0%
April 24, 2026	8	20	40.0%
April 27, 2026	4	56	7.1%
Total	52	308	16.9%

Table 8: Daily clear-table task-level success rates from late-stage physical logs.

The day-to-day five-day diagnostic subset also shows that reliability was unstable rather than steadily solved. The best single-day performance in that subset was April 24, with 8 successes in 20 attempts (40.0%), while April 21 and April 27 were below 8%. This spread suggests that calibration state, object placement, scene alignment, tag visibility, and runtime conditions substantially affected performance.

Clear-Table Metric	Count	Rate
Strict full-task successes, final seven-day window	52	16.9%
Successful object moves	100	32.5%
Normal task-completion lines, five-day diagnostic subset	30	9.7%
No target detected after scan	28	9.1%
Full completions, five-day diagnostic subset	98	31.8%
Reached normal completion at all	152	49.4%

Table 9: Clear-table step-level outcomes from late-stage physical logs.

These results indicate that the clear-table task was partially functional but not reliable end-to-end. The most common problems occurred after detection, during grasp execution, object transport, and later-stage object handling.

This supports the conclusion that the major barrier was not simply finding objects, but maintaining sufficient pose accuracy, collision-scene fidelity, and grasp consistency to complete the full manipulation sequence.

D. Pick Dropped Bottle Results

The dropped-bottle workflow remained much less reliable than the tabletop manipulation pipeline. In the available late-stage bottle logs, 38 bottle-pick starts were observed, with 2 logged successful completions, corresponding to a strict task-level success rate of 5.3%. This task was especially sensitive to floor-level reachability, low-height grasp geometry, gripper orientation, and collision constraints around the wheelchair, floor, and robot body.

Despite the low full-task success rate, partial runs showed that important intermediate functions were present. In some runs, the bottle-detection and approach pipeline executed far enough to support grasp attempts and limited transport behavior. However, these partial successes were not repeatable enough to support a strong claim of reliable dropped-object retrieval. The bottle task should therefore be presented as an incomplete but partially demonstrated capability.

Compared with Clear Table, the dropped-bottle task introduced a more difficult physical workspace. The arm had to reach downward toward the floor while maintaining a feasible wrist orientation and avoiding collision with the floor, table, wheelchair, and its own body. These constraints made the task more sensitive to small calibration errors and made recovery more difficult after a failed approach or grasp attempt. The observed result supports the conclusion that floor retrieval should be treated as a future extension after the tabletop manipulation loop becomes repeatable.

E. Medication Hand-Off Results

The medication hand-off workflow demonstrated the intended safety-oriented interaction structure, but did not achieve a defensible end-to-end physical success result in the late-stage testing window. Only 3 medication command starts were identified in the same five-day span, and no clear end-to-end success marker was found. For that reason, a task-level success rate is not reported for this workflow.

Step-level evidence shows that parts of the medication pipeline did operate. The logs showed activation of the medication task, scene scanning, QR-read positioning attempts, and patient-name confirmation input from the UI in at least some runs. This indicates that the perception and interaction logic were partially operational. The remaining failures occurred during manipulation setup and collision-constrained motion near the pick phase, reinforcing that the main limitation was not purely user interaction or tag recognition, but full pipeline integration through grasp execution and motion feasibility.

Date	Give Med Starts	Full Successes	Strict Success Rate
April 29, 2026	67	10	14.90%
April 30, 2026	13	3	23.1%
Total	80	13	16.3%

Table 10: Task-level physical results from late-stage command logs.

The medication task showed the strongest improvement after the UI verification stage. On April 30, 4 trials reached patient-name input from the UI, and 3 of those completed hand-off, corresponding to a post-verification success rate of 75.0%. This suggests that once the task reached the user-confirmation stage, the later build was often able to complete the remaining hand-off sequence. However, the strict command-start success rate remained limited because many command starts still did not reach or pass every required stage.

Date	Give Med Starts	Full Successes
April 29, 2026	67	10
April 30, 2026	13	3
Total	80	13

Table 11: Give Medication step-level result after patient-name verification.

These results show that Give Medication should no longer be described as having no confirmed end-to-end success. The more accurate conclusion is that the task remained unreliable at the strict command-start level but demonstrated meaningful late-stage improvement, especially after the verification step. Remaining failures were concentrated around QR-read positioning, medication grasp-pose reachability, incomplete terminal markers, and earlier pipeline stages before patient-name verification.

F. Observed Failure Modes

The dominant issues were integration failures rather than isolated single-module failures. The most important observed failure categories were: (1) AprilTag detections had to be transformed from the camera frame into base_link and then converted into object-center poses, where small errors produced inaccurate grasp targets; (2) tags were not always located at the physical center of the object, making side-tagged and cylindrical objects especially sensitive to offset calculations; (3) the static collision scene did not always match the physical table, wheelchair, floor, shelf, bin, and surrounding obstacles; (4) MoveIt 2 rejected or failed plans when requested poses conflicted with collision geometry or workspace limits; (5) the wrist camera and cable introduced physical dependencies that could disturb perception; and (6) the project attempted three distinct ADL workflows before any single task had been made highly repeatable.

The failure patterns differed by ADL. Clear Table most often exposed object-level manipulation and placement inconsistency after partial task progress. Pick Dropped Bottle was most affected by constrained floor-level reachability and grasp geometry. Give Medication was most affected by whether the system could reach the QR-read and medication grasp stages before patient-name verification, but later logs showed improved hand-off behavior once verification was reached.

G. Prototype Status Summary

Overall, the physical results suggest that the primary limitation was not the absence of perception or motion in isolation, but insufficient reliability at the interfaces between

detection, pose transformation, scene modeling, grasp generation, and collision-constrained execution. The project demonstrated an integrated architecture, repeated physical command attempts, partial object manipulation, and late-stage medication hand-off improvement. However, it did not meet the original reliability goals for autonomous ADL execution.

The clearest supported claim is that the system reached the status of an early-stage structured-lab prototype with operational subsystems and measurable partial-task performance. It should not be described as a finished assistive manipulation device or a reliable autonomous ADL system.

VI. DISCUSSION

A. Interpretation of Results

The late-stage results show that the selected architecture was operational enough to support repeated physical command attempts, but not reliable enough for deployable ADL execution. The strongest breadth of testing came from the clear-table workflow, where 538 command attempts were logged across the final seven physical test days. Clear Table achieved 70 strict full-task successes from 538 command starts, or 13.0%, while also achieving 155 successful object moves from 298 object attempts, or 52.0%. This distinction shows that individual object manipulation sometimes worked even when full multi-object ADL completion did not.

The dropped-bottle workflow remained the weakest ADL by strict task-level reliability, with 2 logged completions from 38 starts, or 5.3%. This result reflects the added difficulty of floor-level reachability, low grasp height, and collision constraints. Give Medication improved late in the build, reaching 13 successful hand-offs from 80 command starts across the final two testing days, or 16.3%. On the final day, 3 of the 4 trials that reached UI patient-name input completed hand-off, or 75.0%, indicating that the post-verification hand-off sequence improved even though the full command-to-completion pipeline remained unreliable.

These results clarify the difference between prototype operation and task reliability. In simulation or stub-based testing, object poses and scene geometry can be controlled or simplified. On physical hardware, small errors in camera calibration, tag placement, object geometry, table position, cable movement, or scene modeling can prevent a manipulation sequence from completing. Because each ADL task depended on a long pipeline of perception, transformation, planning, grasping, transport, placement, and recovery, failures accumulated across subsystem boundaries rather than appearing as a single isolated defect.

B. Perception and Calibration Limitations

AprilTag-based perception was selected because it provided a simpler and more deterministic alternative to learned object-detection approaches. This matched the design trade-off described in Section III-D: AprilTags avoid the need for a custom training dataset and allow objects to be identified directly by tag ID [12]–[16]. However, the physical results in Section V show that AprilTag identification alone is not sufficient for reliable manipulation because the robot must also estimate a physically accurate grasp pose.

The robot did not need only to recognize an object; it needed to estimate a physically accurate grasp pose. This required the system to transform the detected tag pose into the robot base frame and then infer the object center, grasp

offset, and approach vector. For top-tagged objects, this relationship was more direct. For side-tagged objects and cylindrical objects, the relationship between tag pose and object center was more sensitive to tag placement, object orientation, and calibration error. These errors contributed to missed grasps, planning-scene mismatch, and failed motion plans.

C. Planning-Scene and Motion-Planning Limitations

The planning scene was necessary for safe operation because the arm operated near a table, wheelchair, floor, and user-adjacent handoff zones. However, the planning scene also became a source of failure when modeled geometry did not exactly match the physical environment described in Section IV-A. A static scene is useful only when the actual workspace remains aligned with the model. If the table, shelf, bin, wheelchair, or camera pose shifts, the robot may reject safe motions, accept unsafe ones, or compute grasp paths that no longer correspond to the real setup.

Motion planning also became more difficult near the boundaries of the workspace. The dropped-bottle task was especially constrained because the arm had to reach near the floor while avoiding collisions and preserving a feasible gripper orientation. The tabletop tasks were less extreme but still sensitive to table height, object placement, and gripper-clearance assumptions. These results suggest that future work should begin with a smaller validated workspace and a single repeatedly tested object before adding multi-object clearing, medication delivery, and floor retrieval.

D. Scope Trade-Off

The project attempted to implement three distinct ADL workflows: clear table, pick dropped bottle, and medication hand-off. This choice demonstrated intended versatility and justified the modular structure in Section III, but it also spread development time across multiple task types before any single task had been made highly reliable. The quantitative results in Section V support this scope trade-off. The most-tested workflow, Clear Table, reached 70 strict full-task successes from 538 command starts, or 13.0%, although object-level performance was higher at 155 successful object moves from 298 attempts, or 52.0%. Pick Dropped Bottle reached only 2 logged successful completions from 38 starts, or 5.3%. Give Medication improved during final testing, reaching 13 successful hand-offs from 80 command starts, or 16.3%, with a final-day post-verification rate of 3/4, or 75.0%. These values show that the modular architecture supported partial progress across multiple ADLs, but development time was spread across several different manipulation problems before any one task reached reliable repeatability.

In hindsight, a narrower scope would likely have produced stronger physical results. Focusing first on one tabletop object could have allowed more time for perception calibration, grasp tuning, placement validation, emergency-stop testing, and repeated trials. Once that task was reliable, the architecture could then expand to multi-object clearing, medication handling, and floor retrieval. The final prototype therefore shows the value of modular design but also shows that modularity does not replace the need for deep task-specific calibration.

E. Safety and Assistive Use

Because the arm operates near a user, the system must be evaluated under stricter safety expectations than a non-assistive lab robot. The current prototype included several safety-oriented mechanisms, including conservative motion limits, collision-aware planning, status reporting, recovery poses, emergency-stop behavior, and medication name verification. These design choices align with the safety concerns identified in Section III-A and with relevant robot and machinery safety standards [22]-[24].

A deployable assistive system would need reliable task execution, repeated emergency-stop validation, robust user-interface testing, failure recovery, physical cable management, and evaluation with representative users. The current system should therefore be described as an early-stage structured-lab prototype rather than a certified or user-ready assistive device. This distinction also limits the accessibility claim in Section III-C: the UI is accessibility-oriented, but it was not user-validated.

F. Lessons Learned and Future Work

The most important lesson from the project is that robotic ADL manipulation is primarily an integration problem. Individual components can appear functional in isolation while the full pipeline remains unreliable. In this project, step-level progress was visible in the clear-table logs, but full-task reliability remained low because the camera must remain fixed and calibrated, the AprilTag must be visible, the tag pose must transform correctly, the object-center offset must be accurate, the planning scene must match the physical workspace, MoveIt must find a feasible collision-free path, the gripper must align correctly, and the object must remain stable during transport and release.

Future work should prioritize reliability over feature expansion. The first recommended step is to stabilize one tabletop pick-and-place task with a single object. This should include repeated tag-pose measurements, manual verification of object-center offsets, repeated grasp attempts, and controlled placement trials. Specific improvements include better wrist-camera cable management, verified camera-to-tool transforms, object-specific calibration procedures, reduced reliance on hardcoded scene assumptions, per-stage failure logging, emergency-stop validation across every task stage, and user testing of the interface with users who have reduced upper-limb mobility.

VII. CONCLUSION

This work presented a ROS 2-based wheelchair-mounted assistive robotic arm prototype for structured Activities of Daily Living. The system integrated a Kinova Gen3 7-DoF manipulator, Robotiq 2F-85 gripper, wrist-mounted Intel RealSense camera, AprilTag-based perception, MoveIt 2 motion planning, and a tablet-accessible user interface [6]-[10], [12]-[16]. Three ADL workflows were implemented in software: clearing tabletop objects, retrieving a dropped water bottle, and medication hand-off with patient-name verification.

Late-stage physical testing produced measurable but limited task-level results. Across the seven physical test days used for final clear-table analysis, the clear-table workflow achieved 70 strict full-task successes in 538 command attempts, corresponding to a task-level success rate of 13.0%. Its object-level performance was higher, with 155 successful

object moves from 298 object attempts, or 52.0%, showing that partial manipulation was more reliable than full workflow completion. The dropped-bottle workflow achieved 2 logged successful completions in 38 starts, or 5.3%, and remained the least reliable ADL due to floor-level reachability and grasp constraints. The medication hand-off workflow improved during the final two days of testing, achieving 13 successful hand-offs from 80 command starts, or 16.3%, with a final-day post-verification hand-off rate of 3/4, or 75.0%. These results show that the system was not reliable enough to be described as a completed assistive device, even though several subsystems and intermediate execution stages were operational.

The primary contribution of the project is therefore an integrated prototype and failure analysis rather than a finished ADL product. The results identify the main remaining barriers as perception-to-motion calibration, object-center estimation, grasp-position offsets, planning-scene mismatch, constrained workspace geometry, and physical camera/cable reliability. Future work should narrow the scope to one repeatable tabletop pick-and-place task, collect controlled stage-level logs, improve calibration and cable management, validate emergency-stop behavior across every stage, and expand to floor retrieval and medication hand-off only after the basic manipulation loop is reliable.

VIII. ACKNOWLEDGMENT

We would like to thank the USF CARRT Lab, and specifically Dr. Alqasemi and Sohrab Moayer for advice and guidance during the development of the system. Without your advice and expertise, the creation of the system would not have been possible.

REFERENCES

- [1] A. NANAVATI, V. RANGANENI, AND M. CAKMAK, "PHYSICALLY ASSISTIVE ROBOTS: A SYSTEMATIC REVIEW OF MOBILE AND MANIPULATOR ROBOTS THAT PHYSICALLY ASSIST PEOPLE WITH DISABILITIES," ANNUAL REVIEW OF CONTROL, ROBOTICS, AND AUTONOMOUS SYSTEMS. [ONLINE]. AVAILABLE: [HTTPS://PMC.NCBI.NLM.NIH.GOV/ARTICLES/PMC12978752/](https://PMC.NCBI.NLM.NIH.GOV/ARTICLES/PMC12978752/). [ACCESSED: APR. 2, 2026].
- [2] M. S. H. Sunny et al., "Eye-gaze control of a wheelchair mounted 6DOF assistive robot for activities of daily living," *Journal of NeuroEngineering and Rehabilitation*. [Online]. Available: <https://link.springer.com/article/10.1186/s12984-021-00969-2>. [Accessed: Apr. 2, 2026].
- [3] M. Chi et al., "Development and evaluation of demonstration information recording approach for wheelchair mounted robotic arm," *Complex & Intelligent Systems*. [Online]. Available: <https://link.springer.com/article/10.1007/s40747-021-00350-9>. [Accessed: Apr. 2, 2026].
- [4] S. Jain et al., "Assistive robotic manipulation through shared autonomy and a body-machine interface," in *Proc. IEEE International Conference on Rehabilitation Robotics*. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC4737957/>. [Accessed: Apr. 2, 2026].
- [5] M. T. Shahria, J. Ghommam, R. Fareh, and M. H. Rahman, "Vision-based object manipulation for activities of daily living assistance using assistive robot," *MDPI*. [Online]. Available: <https://www.mdpi.com/2673-4052/5/2/6>. [Accessed: Apr. 2, 2026].
- [6] Kinova Robotics, "KINOVA Gen3 ultra lightweight robot user guide," 2022. [Online]. Available: <https://www.kinovarobotics.com/uploads/User-Guide-Gen3-R07.pdf>. [Accessed: Apr. 9, 2026].
- [7] Robotiq Inc., "2F-85 and 2F-140 adaptive robot grippers: Specifications," *Robotiq Documentation*. [Online]. Available: <https://robotiq.com/products/2f85-140-adaptive-robot-gripper>. [Accessed: Apr. 9, 2026].
- [8] Intel Corporation, "Intel RealSense Depth Camera D435," *Intel Product Specifications*. [Online]. Available: <https://www.intel.com/content/www/us/en/products/sku/128255/intel-realsense-depth-camera-d435/downloads.html>. [Accessed: Apr. 9, 2026].
- [9] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022, doi: 10.1126/scirobotics.abm6074.
- [10] PickNik Robotics, "MoveIt 2 Documentation," 2025. [Online]. Available: <https://moveit.picknik.ai/main/index.html>. [Accessed: Apr. 9, 2026].
- [11] Kinova Robotics, "ros2_kortex: ROS 2 driver for Kinova Gen3 robot arm," *GitHub repository*, 2026. [Online]. Available: https://github.com/Kinovarobotics/ros2_kortex. [Accessed: Apr. 9, 2026].
- [12] E. Olson, "AprilTag: A robust and flexible visual fiducial system," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2011, pp. 3400-3407, doi: 10.1109/ICRA.2011.5979561.
- [13] J. Wang and E. Olson, "AprilTag 2: Efficient and robust fiducial detection," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 4193-4198, doi: 10.1109/IROS.2016.7759617.
- [14] M. Krogus, A. Haggemiller, and E. Olson, "Flexible layouts for fiducial tags," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 1898-1903, doi: 10.1109/IROS40897.2019.8967787.
- [15] AprilRobotics, "AprilTag 3," *GitHub repository*, 2025. [Online]. Available: <https://github.com/AprilRobotics/apriltag>. [Accessed: Apr. 9, 2026].
- [16] Pupil Labs GmbH, "pupil-apriltags documentation," 2026. [Online]. Available: <https://pupil-apriltags.readthedocs.io/en/stable/>. [Accessed: Apr. 9, 2026].
- [17] OpenCV, "Detection of ArUco markers," *OpenCV 4.x Documentation*, 2026. [Online]. Available: https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html. [Accessed: Apr. 9, 2026].
- [18] E. Fernandez, "ROS 2 Quality of Service policies," *ROS 2 Design Docs*, 2019. [Online]. Available: <https://design.ros2.org/articles/qos.html>. [Accessed: Apr. 9, 2026].
- [19] T. Foote and M. Purvis, "REP 0103: Standard units of measure and coordinate conventions," *Open Robotics*, 2010. [Online]. Available: <https://reps.openrobotics.org/rep-0103/>. [Accessed: Apr. 9, 2026].
- [20] W. Meeussen, "REP 0105: Coordinate frames for mobile platforms," *Open Robotics*, 2010. [Online]. Available: <https://reps.openrobotics.org/rep-0105/>. [Accessed: Apr. 9, 2026].
- [21] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.2," *W3C Recommendation*, 2024. [Online]. Available: <https://www.w3.org/TR/WCAG22/>. [Accessed: Apr. 27, 2026].
- [22] International Organization for Standardization, "ISO 10218-1:2011: Robots and robotic devices -- Safety requirements for industrial robots -- Part 1: Robots," 2011. [Online]. Available: <https://www.iso.org/standard/51330.html>. [Accessed: Apr. 27, 2026].
- [23] International Organization for Standardization, "ISO/TS 15066:2016: Robots and robotic devices -- Collaborative robots," 2016. [Online]. Available: <https://www.iso.org/standard/62996.html>. [Accessed: Apr. 27, 2026].
- [24] International Electrotechnical Commission, "IEC 62061:2021: Safety of machinery -- Functional safety of safety-related control systems," 2021. [Online]. Available: <https://webstore.iec.ch/publication/59927>. [Accessed: Apr. 27, 2026].
- [25] Open Robotics, "Gazebo Sim documentation," 2026. [Online]. Available: <https://gazebo.org/docs/>. [Accessed: Apr. 27, 2026].

APPENDIX A

High-Level Diagram of the CARRT Assistive Arm

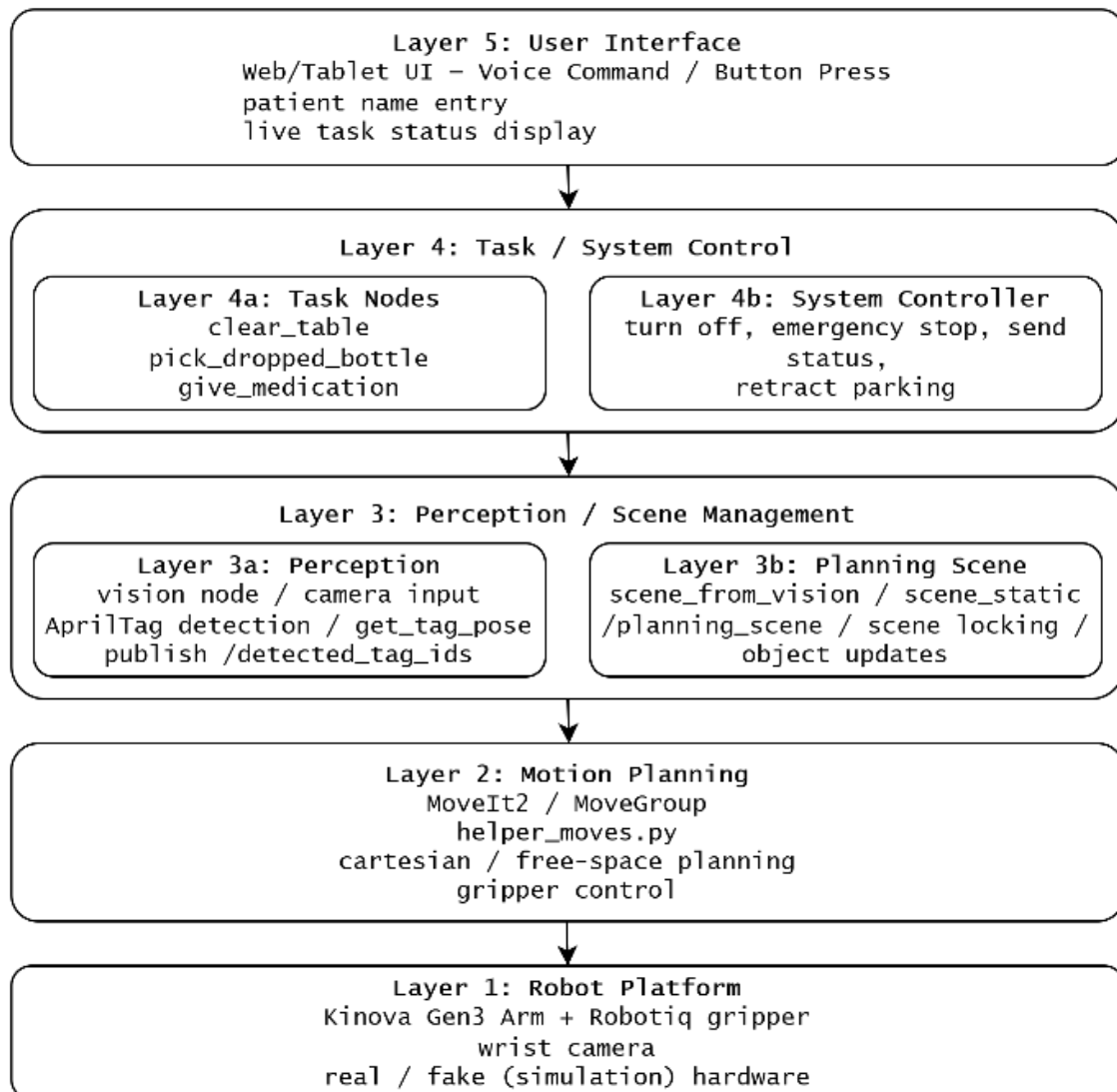


Figure 4: High-Level Diagram of the ADL Support System

ADL Task Execution Pipeline

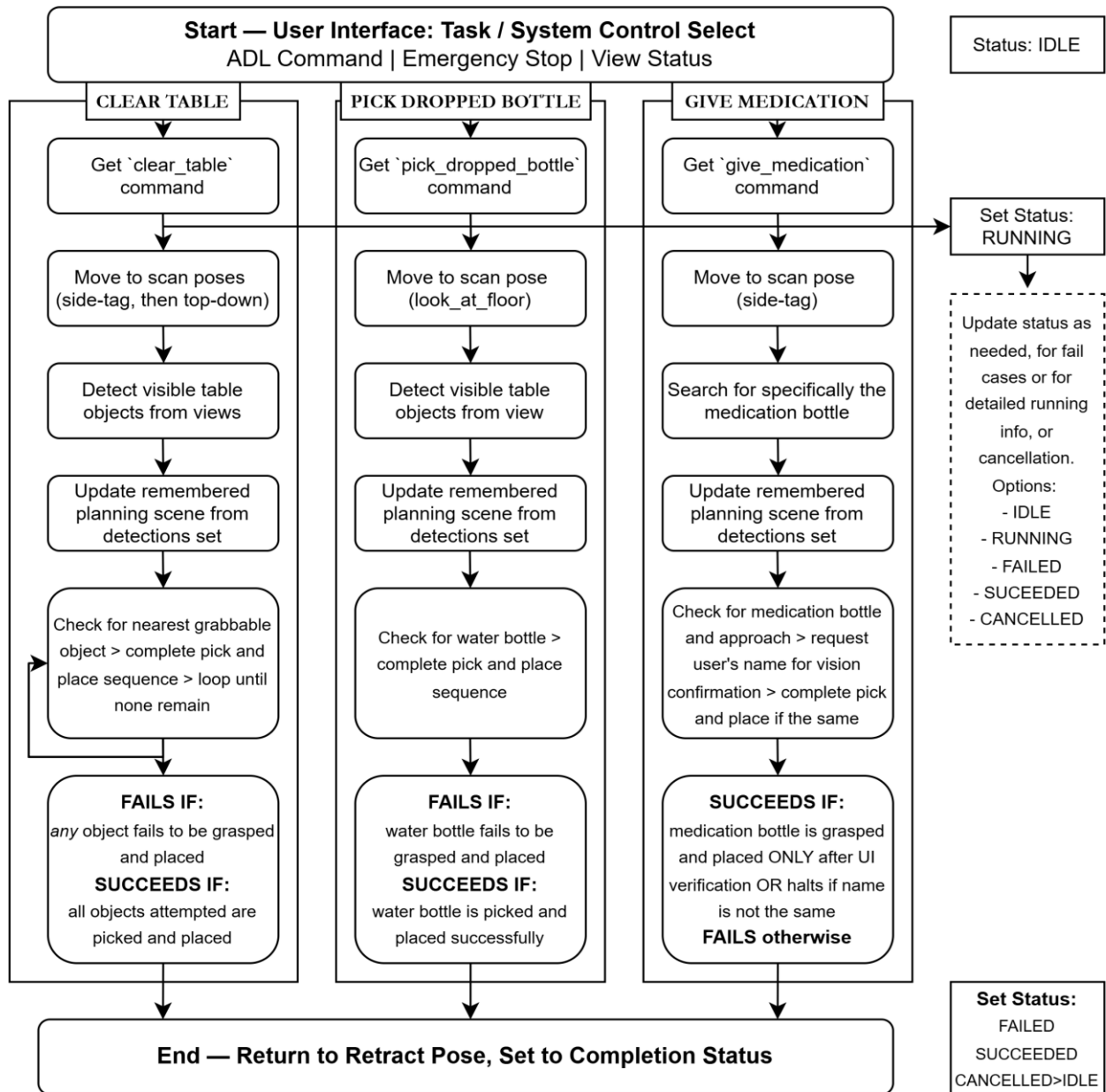


Figure 5: Execution Pipeline of Each ADL Task